# SandEE: AN EDUCATIONAL MULTI-PLATFORM 2D OPEN WORLD SURVIVAL CRAFTING GAME FOR LUA AND PYTHON FOR ICT AND STEM STUDENTS OF YVERDON DE PESTALOZZI SCHOOL INC.

**A Thesis**
**Presented to the Faculty of the**
**Information and Communications Technology Program**
**STI College San Jose Del Monte**

**In Partial Fulfilment**
**of the Requirements for the Degree**
**Bachelor of Science in Computer Science**

**Glenn Marco B. Accad**
**Adrian V. Gallano**
**Jefferson G. Mamauag**
**Jose Mari Christian V. Tayag**

**November 2026**

**ENDORSEMENT FORM FOR ORAL DEFENSE**

**TITLE OF RESEARCH:** SandEE: An Educational Multi-Platform 2D Open World Survival Crafting Game for Lua and Python for ICT and STEM Students of Yverdon de Pestalozzi School Inc.

**NAME OF DEVELOPERS:** Glenn Marco B. Accad
Adrian V. Gallano
Jefferson G. Mamauag
Jose Mari Christian V. Tayag

In Partial Fulfilment of the Requirements
for the degree Bachelor of Science in Computer Science
has been examined and is recommended for Oral Defense.

**ENDORSED BY:**

Norvin Shaq F. Lintag
**Thesis Adviser**

**APPROVED FOR ORAL DEFENSE:**

Enoch James C. Tayanguna
**Thesis Coordinator**

**NOTED BY:**

Shiela Marie B. Llesol
**Program Head**

**November 2026**

## APPROVAL SHEET

**This thesis titled SandEE: An Educational Multi-Platform 2D Open World Survival Crafting Game for Lua and Python for ICT and STEM Students of Yverdon de Pestalozzi School Inc., prepared and submitted by Glenn Marco B. Accad; Adrian V. Gallano; Jefferson G. Mamauag; and Jose Mari Christian V. Tayag, in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science, has been examined and is recommended for acceptance and approval.**

**Norvin Shaq F. Lintag**
Thesis Adviser

Accepted and approved by the Thesis Review Panel
in partial fulfillment of the requirements for the degree of
Bachelor of Science in Computer Science

**Panel Member** **Panel Member**

**Lead Panelist**

**APPROVED:**

Enoch James C. Tayanguna
**Thesis Coordinator**

Shiela Marie B. Llesol
**Program Head**

**November 2026**

## ACKNOWLEDGEMENTS

The researchers would like to express their sincere gratitude to the individuals and institutions who made this study possible.

To their thesis adviser, Mr. Norvin Shaq F. Lintag, for his guidance, patience, and constructive feedback throughout the planning, development, and writing of this study;

To the thesis coordinator, Mr. Enoch James C. Tayanguna, for his support in the coordination, review, and defense process of the study;

To the members of the Thesis Review Panel, Mr. Louis Matthew P. Sindao and Mr. Lianoel Vergante, for their valuable comments and suggestions that helped improve the quality of this work;

To the faculty and staff of the Information and Communications Technology Program, for providing the academic environment and resources needed to complete this study;

To the Grade 11 and 12 ICT and STEM students of Yverdon de Pestalozzi School Inc., for their participation in the needs assessment and evaluation of SandEE;

To their parents, guardians, and families, for their unwavering love, patience, and moral support; and

To their friends, classmates, and everyone who contributed in one way or another to the completion of this study, the researchers offer their heartfelt thanks.

## Abstract

**Title of research: SandEE: An Educational Multi-Platform 2D Open World Survival Crafting Game for Lua and Python for ICT and STEM Students of Yverdon de Pestalozzi School Inc.**

Researchers: Glenn Marco B. Accad, Adrian V. Gallano, Jefferson G. Mamauag, and Jose Mari Christian V. Tayag

Degree: Bachelor of Science in Computer Science

Date of Completion: November 2026

Keywords: game-based learning, Lua, Python, educational game, programming education, survival crafting, open-world game

---

This study developed SandEE, an educational multi-platform 2D open-world survival crafting game designed to supplement the learning of Lua and Python among Grade 11 and 12 ICT and STEM students of Yverdon de Pestalozzi School Inc. The study was motivated by a needs assessment survey of 104 students, which showed limited familiarity with Lua but strong interest in learning it through interactive, game-based methods. Using the Waterfall development model, the researchers designed and developed a single-player game that combines procedural world generation, survival and crafting mechanics, mission-based progression, and embedded Lua and Python interpreters. Player-written programs control LUA robots and Algorithmica stations, allowing code to affect the game world in real time; syntax and runtime errors are reported in the in-game editor so debugging becomes part of the learning process. The game also includes themed Lua and Python dungeons, knowledge and quiz altars, a programming encyclopedia, and optional online accounts, cloud saves, and leaderboards. The system was evaluated through testing with IT students and expert review using selected ISO 25010 software quality characteristics. The evaluation produced an overall weighted mean of 3.61, interpreted as Very High. The results indicated that the game met its functional requirements and was rated favorably in terms of functional suitability, performance efficiency, and usability. The study concludes that SandEE can serve as a supplementary learning tool that makes programming practice more interactive, contextual, and engaging for beginner learners. Recommendations for further development include additional missions, classroom integration, accessibility improvements, and a longer-term study of learning outcomes.

## LIST OF TABLES


1 Functional Modules and Descriptions ..... 48
2 Hardware ..... 60
3 Software ..... 61
4 Budgetary Estimate ..... 64
5 Mean Range Interpretation Legend ..... 65
6 Functional Completeness ..... 68
7 Functional Correctness ..... 69
8 Functional Appropriateness ..... 69
9 Time Behaviour ..... 70
10 Resource Utilization ..... 70
11 Capacity ..... 71
12 Appropriateness Recognizability ..... 71
13 Learnability ..... 72
14 Operability ..... 72
15 User Error Protection ..... 73
16 User Interface Aesthetics ..... 73
17 Accessibility ..... 73
18 Summary of IT Student Evaluation Results ..... 74
19 Actual Thesis Expenses ..... 88
20 SandEE Controls ..... 91

## LIST OF FIGURES


1 Waterfall Methodology ..... 44
2 Conceptual Framework ..... 46
3 System Architecture ..... 49
4 Data Flow Diagram Level 0 ..... 52
5 Data Flow Diagram Level 1 ..... 53
6 Entity Relationship Diagram ..... 54
7 Hierarchical Chart ..... 55
8 Account Authentication Flowchart ..... 56
9 Updated Gameplay Flowchart ..... 57
10 Dungeon Gameplay Flowchart ..... 58
11 SandEE Gameplay Screenshots ..... 65

## LIST OF APPENDICES


A Gantt Chart ..... 84
B Actual Thesis Expenses ..... 87
C User's Manual ..... 89
D Survey ..... 93
E Curriculum Vitae of Researchers ..... 103

## TABLE OF CONTENTS

**Acknowledgements** **iv**
**Abstract** **v**
**List of Tables** **vi**
**List of Figures** **vii**
**List of Appendices** **viii**
**Table of Contents** **ix**
**Introduction** **11**
Background of the problem ..... 13
General Problem ..... 14
Specific Problem ..... 14
Overview of the current state of the technology ..... 16
Objectives of the study ..... 19
General Objectives ..... 19
Specific Objectives ..... 19
Scope and limitations of the study ..... 21
Review of related literature/studies/systems ..... 35
Foreign Literature ..... 35
Local Literature ..... 36
Foreign Studies ..... 37
Local Studies ..... 39
Foreign Systems ..... 40
Local Systems ..... 41
Synthesis ..... 42
**METHODOLOGY** **44**
Conceptual Framework ..... 46
System Design Specification ..... 47
Diagrams ..... 51
Hardware/Software ..... 60
Calendar of Activities ..... 62
Budgetary Estimate ..... 64

**Results and Discussions** **65**
Overview of the results ..... 65
System Implementation ..... 65
Data Gathering Procedure ..... 66
Statistical Treatment of Data ..... 67
User Feedback ..... 68
Summary of IT Student Evaluation Results ..... 74
Expert Feedback ..... 75
**Conclusion** **76**
Summary ..... 76
Conclusions ..... 77
Recommendations ..... 79
**References** **81**
APPENDIX A. GANTT CHART ..... 84
APPENDIX B. ACTUAL THESIS EXPENSES ..... 87
APPENDIX C. USER'S MANUAL ..... 89
APPENDIX D. SURVEY ..... 93
APPENDIX E. CURRICULUM VITAE OF RESEARCHERS ..... 103

# INTRODUCTION

In recent times, especially in information communication technology, programming has been the focus of computing education. The recent advent of several tools has made coding more accessible than ever (Becker, 2023). Programming has been a challenge for learners. The proliferation of digital technologies has made programming skills increasingly important across various disciplines. This shift extends beyond traditional computer science domains (Lin, et.al., 2025). According to industry reports, the demand for Lua developers has been steadily increasing in recent years. As more companies turn to Lua for its speed, simplicity, and versatility, the need for skilled Lua developers has never been higher. In particular, the gaming industry has seen a rise in Lua-related job openings, with many companies looking for developers proficient in Lua scripting (Crudu, 2024). The Lua scripting language is a fast, procedural programming language it was designed to be highly customizable with simple API in the C programming language so that programmers could easily make changes to fit their needs. Python has a simple syntax that allows developers to write programs with fewer lines compared to other programming languages Additionally, Python as a scripting language has been steadily gaining traction since its inception. Its simplified syntax allows learners to easily learn and grasp the coding structure of the scripting language (Python, 2021).

Yverdon De Pestalozzi School Inc. is a private senior high school offering STEM and ICT strands, dedicated to developing students' technical skills, academic competence, and readiness for future careers through a supportive and well-structured learning environment.

According to Lampropoulos (2023), Game-based learning emerged as a useful education approach that can be facilitated and enrich the learning process especially in students. This approach creates a motivational virtual learning environment that consciously engages and encourages students to actively participate in educational activities. This approach has also been used extensively in the field of programming, allowing an alternative approach to learning various programming concepts.

In a survey conducted to ICT and STEM Senior High School students of Yverdon De Pestalozzi School Inc., a total of 104 respondents participated, consisting of 22 students from Grade 11 ICT (21.2%), 32 from Grade 11 STEM (30.8%), 16 from Grade 12 ICT (15.4%), and 34 from Grade 12 STEM (32.7%). The results revealed that most students demonstrated average familiarity with the Lua programming language, suggesting that it is relatively new to many respondents. In addition to this, the majority expressed positive attitudes toward learning Lua through supplementary game-based activities, with 81 (78.9%) agreeing that such an approach would improve their understanding of programming concepts and 76 (73%) showing motivation to explore Lua using an interactive approach. On the other hand, Python received generally favorable feedback, with 84 (80.8%) of participants finding its syntax and structure easy to understand and applicable to programming tasks. Overall, the findings indicate that while students are less experienced with Lua compared to Python, they show strong enthusiasm and willingness to learn it through interactive and game-oriented methods that enhance engagement and learning outcomes. As a result, introducing a gamified approach of learning Lua and Python is applicable to the students at the school.

SandEE is an educational 2D open world survival crafting game for Lua and Python. It introduces several programming concepts related to Lua and Python. Furthermore, SandEE also includes gameplay that can serve as an exercise to advanced users of Lua and Python. The proposed study offers a supplementary learning approach for Lua and Python by slowly introducing its syntax and structure. The game embeds interpreters for Lua and Python in which code that is run impacts the game environment to show visual feedback, as well as to provide a realistic basis to the player as the interpreters will give information accurate to its real use-case. Progression is achieved through the mission system, which guides the player to progress in learning. These missions will eventually lead to rewards to further gratify the player and give them feedback on their progress.

**Background of the problem**

The rapid advancement of technology has significantly transformed educational methodologies. Traditional learning, which often emphasizes theoretical concepts and static exercises, struggles to fully engage students or prepare them for real-world problem-solving. In contrast, educational games have emerged as an innovative approach to learning, combining interactivity and engagement with skill development. Popular examples like Duolingo demonstrate how gamified systems can make learning both effective and enjoyable. Game-based learning in programming has shown the potential to enhance logical reasoning, problem-solving, and creativity through active participation.

To better understand the learning preferences of ICT and STEMS students, a survey was conducted among the grade 11 and 12 ICT and STEM students of Yverdon De Pestalozzi School Inc. The results revealed that most respondents had average familiarity with the Lua programming language, with the majority rating their understanding between 2 and 4 on a 4-point scale. However, despite this lack of experience, 82 (78.9%) of respondents rated 2, 3, and 4 when asked how helpful and motivating it would be to learn Lua through game programming. For comparison, students showed higher familiarity with Python, averaging between 2, 3 and 4 in both ease of use and coding level. Yet even those comfortable with Python expressed strong interest in learning programming concepts through interactive, game-based methods. These findings indicate that students are more receptive and motivated when learning involves playing, which shows what happens inside the game, experimentation, and engagement rather than passive listening and reading books.

To address these findings, the proposed game introduces an immersive, game-based platform that integrates survival and crafting mechanics within an open-world sandbox-type environment. Through hands-on coding challenges embedded in gameplay, it aims to foster critical thinking, logical reasoning, and practical programming skills. This approach seeks to make programming educational, creative, experiential, contextual, and enjoyable.

**General Problem**

**How to develop a game to supplement the high interest in learning Lua and Python through a game?**

*Despite low familiarity with ICT and STEM students, survey results have shown that interest in learning Lua and Python is high. Furthermore, the results have also shown an increasing interest in learning these languages using a more interactive and engaging approach through a game.*

**Specific Problem**

- **How to develop a game that enhances programming skills by providing hands-on learning where concepts are learned through experience?**

  *Many ICT and STEM students have shown a strong interest in learning programming languages such as Lua and Python, particularly through interactive and engaging methods. Based on the survey conducted, there is a clear inclination toward learning through game-based experiences that allow the practical use of programming concepts. Therefore, this study focuses on developing a game that enhances programming skills by providing hands-on learning where concepts are learned through experience, serving as an innovative approach to support growing interest in programming.*

- **How to develop a game that provides direct visual feedback as players learn and apply programming concepts?**

  *Students often show greater engagement and understanding when they can immediately see the results of their actions while learning. In programming education, providing direct visual feedback allows learners to observe how their inputs affect outcomes in real time, creating a more interactive and immersive experience. Therefore, this study aims to develop a game that provides direct visual feedback as players learn and apply programming concepts, promoting better comprehension and active participation through visual interaction.*

- **How to develop a game that realistically responds to player errors by embedding Lua and Python interpreters within the system?**

  *To create a more authentic and educational programming experience, it is important for a game to respond realistically to player inputs, including coding errors. Embedding Lua and Python interpreters within the system enables real-time evaluation of player code, allowing the game to simulate how actual programming environments handle mistakes.*

- **How to develop a game that introduces the programming structure and syntax of Lua and Python?**

  *Learning programming languages such as Lua and Python requires a clear understanding of their syntax and structural rules. Introducing these foundational concepts through an interactive approach can make learning more engaging and effective.*

## Overview of the current state of the technology

In recent years, advancements in teaching coding through educational and engaging means have been accomplished. There are ways, including game-based learning and digital courses, that provide plenty of valuable knowledge on how to code to solve problems and perform tasks.

### Philippine Coding Camp (2023)

Such examples include the Philippine Coding Camp. These are digital learning bootcamps that primarily focus on teaching students of any age by making the courses simpler, welcoming, and open to beginners and experienced learners alike. Both provide structured courses that teach the fundamentals and concepts of programming, which they can later apply to what they have learned through exercises and challenges. They use platforms such as Roblox as a supplement and make learning more engaging. As learners progress through the courses, new concepts are slowly and simply introduced for them to better understand and retain what they are learning.

While these courses are structured, there are limitations that can be addressed. Such as the focus is only centered on the topics of the course. By making the process more hands-on, it removes the restriction of complying with the structure of the course and letting the learners learn in their own way with similar guidance.

**Logiscool (2025)**

Logiscool is a similar coding camp established worldwide and has a digital platform where they also teach digital courses. They provide programs where in each, they teach digital literacy, AI, and coding. They integrate gaming platforms such as Roblox and Minecraft into their learning curriculums to make learning more interesting. In addition, they also use artificial intelligence to support teaching. Part of the programs they teach are emerging languages such as Python and Lua. Python is taught within topics such machine learning, and Lua is primarily used in Roblox to create games and experiences. Logiscool's main target is younger users but are still open to everyone.

Logiscool's programs thoroughly teach its learners about the topics, provide and hones skills such as critical thinking and creativity. But by creating a more engaging way to teach, similarly to how they use games, learners can better learn and apply their skills into practical applications.

**Hello World (2024)**

Hello World is a game that primarily teaches Java and C#. It teaches fundamental concepts of these two-programming languages and integrates them into gameplay. Throughout Hello World, introductory lessons are provided, then the player must use what they have learned to progress. The game then adds the challenge of using different concepts to solve a given problem. It teaches through direct engagement with languages since it is incorporated into the gameplay. The strength of the system comes from the interactive process of learning languages.

This study provides aims to recreate the process of learning through interaction with languages, but uses two emerging languages, Lua and Python. Java and C# are already commonly taught in the field, but there is a growing need in the industry that seeks developers with knowledge of Lua and Python.

**Code Combat (2025)**

CodeCombat is a digital gaming platform that primarily focuses on teaching fundamentals of programming and applying it into game scenarios. Ozaria is built from CodeCombat, where Python is one of the main focuses. They have a custom built-in engine and interpreter that helps beginners in learning how to code. This platform can be integrated in school settings, allowing students to share their projects with others and teachers to grade their work. CodeCombat works by teaching students first, then placing them in game scenarios where their knowledge is put to the test. Ozaria expands this by creating a storyline that progresses with the learner's skill and knowledge on the subject.

There are limitations to this, however. While this type of learning is more engaging, it lacks the structured format that courses provide. They teach first then send the learners into performing tasks. The study aims to bridge the gap between digital courses and gamified learning by blending the two ways of learning into a consolidated process. By merging the structured format of courses and the hands-on engagement that games provide, this creates a foundation for students to learn programming concepts in a unified form.

**CodinGame (2025)**

CodinGame is an online gaming platform that uses coding as its main method of playing. It challenges developers to use their skills in competing against others across different types of games. It encourages the application of what the user knows about programming. It features several programming languages such as Lua and Python, and developers must utilize their skills of the language throughout CodinGame's challenges and competitions. This type of learning helps developers see what they can still learn and improve to solve future challenges and tasks they may face.

CodinGame is limited since it primarily focuses on competition against other players. It doesn't provide an exact way to teach; it only points out what is lacking when it comes to knowledge. This type of learning is limited, hence creating a game that

teaches, and rewards players gives a sense of accomplishment and progress that provides a better experience compared to the competitive nature of CodinGame.

## Objectives of the study

### General Objectives

**To develop a game to supplement the high interest in learning Lua and Python through a game.**

*The proponents of the projects aim to develop a game with the goal of supplementing the high interest of learning the Lua and Python programming language through a game approach by integrating engaging gameplay with learning activities.*

### Specific Objectives

- **To develop a game that enhances programming skills by providing hands-on learning where concepts are learned through experience.**

  *The proponents of the project aim to create an interactive learning environment where students can apply programming concepts directly within gameplay. Specifically, the game will encourage learners to solve coding challenges that simulate real-world scenarios, promote active problem-solving, and experiential learning rather than passive observation.*

- **To develop a game that provides direct visual feedback as players learn and apply programming concepts.**

  *The proponents of the project plan to use the game's environment to display visual feedback based on the user's coding input. Specifically, the player's actions and written code will directly affect the environment, allowing them to see how Lua and Python interpret their logic in real time. This feedback helps learners understand programming flow and identify errors through immediate, intuitive results.*

- **To develop a game that realistically responds to player errors by embedding Lua and Python interpreters within the system.**

  *The proponents of the project aim to ensure that the game reacts authentically to user errors by integrating Lua and Python interpreters. Specifically, this enables the application to detect and display syntax or runtime errors accurately, providing a realistic debugging experience. This approach helps students learn from their mistakes and develop stronger problem-solving and error-handling skills.*

- **To develop a game that introduces the programming structure and syntax of Lua and Python.**

  *The proponents of the project intend to integrate Lua and Python learning modules within the game. Specifically, the application will include coding activities that demonstrate syntax rules, control structures, and logical operations in both languages. This allows users to understand and practice programming concepts naturally while progressing through the game.*

**Scope and limitations of the study**

- **User Account**
    - Each user can create a unique account using their own username and password.
- **Cloud Saves**
    - The save file of each user will be uploaded to the internet every time the game saves.
    - The save file of each user is accessible multi-platform.
    - Cloud saves prioritize the save with the highest amount of stars to prevent conflict.
- **Offline Mode**
    - Allow the player to play the game offline.
    - Players can switch to online mode from offline mode.
- **Online Mode**
    - Allows the player to access the leaderboard and cloud saves.
- **Main Menu**
    - Continue
        - This allows the player to continue from their recently saved world.
    - Worlds
        - This will redirect the user to the saves menu to select their desired save or start a new save.

- Leaderboards (Online Mode Only)
    - List the rankings of player's acquired blue and yellow stars.
- Cloud Save Management (Online Mode Only)
    - Allow the player to manage cloud saves.
- Account (Online Mode Only)
    - Allow overview of the account and allow logging out.
- Settings
    - This will redirect the user to the settings panel to adjust the game's sound volume.
- Quit
    - This will close the game.

- **Settings Panel**
    - Volume Sliders
        - BGM Volume
            - Adjusts the volume for the background music of the game.
        - Effects Volume
            - Adjust the volume for the sound effects of the game.
    - Fullscreen
        - Sets the game to full screen window.
    - Credits

        - Lists the developers and individuals who are part of the game development along with references to the materials and software used.

- **Worlds Menu**
    - World Creation
        - Lets the player name the world before creation.
    - Delete
        - Deletes the chosen save file upon confirmation.
    - Rename
        - Allows the player to rename their worlds.
    - Save Slot
        - Loads the chosen save file to continue its progress.
- **Game Mechanics**
    - Procedural Map Generation
        - Single player maps are automatically created using algorithms instead of being manually designed which allows each environment to be unique.

- Missions
    - A list of gameplay objectives that gradually guides the player throughout the entire game.
    - Dungeon missions are separated into distinct levels of difficulties which encompasses a gradual flow of progression into learning programming.
        - Easy mission introduces basic concepts.
        - Medium missions combine concepts.
        - Hard missions teaches advanced concepts.
    - Completing missions will give the player stars and the stars they have will determine their rank.
- Ranks
    - Ranking system will allow players to display their programming capabilities in the game.
    - Ranks can be increased by completing dungeons and earning blue stars.
- Completion Badge
    - Granted to the player upon completing extra dungeon missions. Only given once.
- Leaderboards
    - Sort available players by their rank.
    - Rankings are based on the blue stars the player has.

- Crafting
    - The user will be crafting various items that improve their current state or progress the missions provided.
    - The user can craft multiple categories of items which include materials, interactive objects, weapons, and foods. Some of which can have additional functionality using programming.
    - The users can craft items to build and progress.
- Fishing
    - Players can use fishing rods to catch fishes in bodies of water.
- Farming
    - Players can use tools to collect seeds and build farmland near water.
- Items
    - Materials
        - It can be used to craft materials and other craftable items.
    - Interactive Objects
        - Crafting Stations
            - Where the user can craft items.
        - Furnace
            - Allows the player to smelt ores and cook food using either wood or charcoal as fuel.

- Chest
    - Allows the player to store items in a limited capacity.
- Color Crafter
    - Allows the player to craft items with custom RGB colors.
    - Displays the hex number for the selected block.
    - Player can select between hex or RGB.

- Weapons
    - Swords can be used to deal damage to enemies.
- Tools
    - Tools such as the pickaxe, axe, and fishing rod can be used for specific purposes.
- Food
    - Can be used to heal health.

- Dungeons
    - Randomly generated areas that serve to give major progress for the user, challenging their skills in Lua or Python programming to unlock more mechanics.
    - Rewards the player with blue stars upon completion.
    - Serves as a guide to player's progress.

        - Randomly spawns successively throughout the world and serve as the dungeon mechanic of the game. The next dungeon spawns after the last one are completed.
        - Includes quizzes that can be solved openly depending on the user.
        - Features Mini-Boss Battle in the last stage.
    - Caves
        - Large Caves
            - Generates in the overworld. Provides finite source of stone and ores.
        - Pixie Caves
            - Special caves with resources that regenerate after a period of time.
        - Friendly Mobs
            - Passive mobs like sheeps and wolves wander the overworld.
    - Enemies
        - Spawns randomly on the overworld and in dungeons.
        - Enemies that spawn on dungeons do not adhere to the day and night cycle.
        - Headless Fish
            - Spawns in the overworld during the night and attacks by homing and dashing into the player.

- Desert Worm
    - Spawns in the overworld regardless of daylight. Spawns and hides in the desert and attack the player when approached.
- Hitman
    - Spawns in the dungeon. Fires projectiles and can charge straight into the player to deal damage.
- Pixie
    - Spawns in the Pixie Caves and attacks the player on sight.

- Day and Night Cycle
    - Determines enemy spawning rate and difficulty.
    - Enemies deal less damage during daytime and do normal damage at night.
- Live Interpreter
    - Serves as a built in IDE for the coding function in the game for both Algorithmica and Automatica.
    - Syntax Highlighting which will highlight the errors in programming syntax.
    - Interprets the code that is written by the player.
- Automatica
    - Robot-like craftables that allow the user to automate certain processes by programming it through Lua, through the Lua-focused dungeons.
    - Automaticas can be used to automate interaction with objects and aid the player in combat.
    - Automaticas required placed wires to interact with stations.
    - Automaticas have the option to be renamed.
- Algorithmica
    - Allows the user to automate actions in crafting stations, furnaces, and color crafters.

- LUA (Live Unit Application)
    - An advanced Automatica non-player character that introduces the user to the Lua language.
- Py-san
    - An advanced Algorithmica non-player character that introduces the user to the Python language.
- Autosave
    - The game will automatically save the player's progress periodically and during state changes such as entering caves and dungeons, as well as when the player quits the game.
- Build Mode
    - Can be toggled on or off to place down blocks.

- **Desktop and Mobile Application**
    - HUD (Heads-Up Display)
        - Pause
            - Allow the user to temporarily pause the game, showing the Pause Menu Panel.
        - Pause Menu Panel
            - Resume
                - Resumes the current game.
            - Settings
                - Shows the settings panel.
            - Lua Management
                - Allows the player to manage the LUA robots.
            - Quit
                - Saves the game and directs the user to the Start Page.
        - Mission Panel
            - Shows the user the current missions they can do to progress.
        - Health Bar
            - Shows the user their current health.
        - Stamina Bar
            - Shows the current stamina, which drains when running.

- Hotbar
    - Show the user their current held items.
- Inventory
    - Shows the user their extra space and a smaller crafting area.
- Programming Encyclopedia
    - Contain details about programming concepts for Lua and Python learned through dungeons and custom functions for Algorithmica and Automatica.
- Code Editor Panel
    - A panel that prompts the user to code in Lua or Python to either progress in a dungeon or customize their Automatica and Algorithmica.
- Crafting Panel
    - A panel for more advanced crafting, unlockable when the user crafts the crafting station.
- Map
    - A representation of the world that shows the overworld, cave, and dungeons.

- On-screen Joystick
    - Allows movement for mobile users.
- Interact Button
    - A general button that changes depending on the items near the player's character.
- Build Mode Button
    - Can be toggled on or off if the player wants to build.

**Limitations**

- **The game only runs on Windows 10 – 11 and Android 8.0 and above.**

  *Ensures the game performs consistently across devices with modern hardware and software support. Older systems may lack the necessary performance, security updates, or engine compatibility required by Godot.*

- **The game only supports Lua and Python for its languages.**

  *This limitation is based on both practicality and accessibility. Lua and Python are in high demand across education, game development, and general programming, making them valuable languages for learners to practice. By restricting the system to these two languages, the game maintains a focused learning environment that is accessible to new programmers while still relevant to real-world applications.*

- **The game only supports single player.**

  *This limitation ensures that each student's learning experience remains individualized and self-paced. By focusing on single-player gameplay the game allows students to develop their skills independently and maintain consistent progress without distractions from multiplayer interactions.*

- **The game only references the 2025 version of official documentation for Lua 5.1 and Python 3.11.**

  *This limitation ensures that all learning materials are consistent, stable, and aligned with a single verified source. Lua 5.1 and Python 3.11 have well-established, mature, and consistent syntax, making them reliable for teaching without frequent changes or breaking updates. By locking the game's references to the 2025 documentation versions, students receive clear and uniform guidance.*

## Review of related literature/studies/systems

### Foreign Literature

According to Crudu (2024), Lua developers are in high demand because it is currently widely used in the programming industry. Lua is efficient, flexible, scalable, and cost effective to apply in any programming field. According to the TIOBE Index, Lua is currently ranked as the 32nd most popular programming language. Furthermore, in a survey conducted by Stack Overflow in 2024 it is among the top 10 most loved programming languages.

According to Becker (2022), Generating high-quality learning resources is very time consuming and typically requires a high level of expertise therefore, games have been used as a learning tool for centuries; they have become more prominent in educational settings. When it comes to learning, being focused and immersed can massively improve a student's experience.

According to Viduka (2021), Python is preferred by students and professionals alike due to its ease of use, availability, efficiency, portability, and extensibility. It is a common choice suitable for education. Many schools starting from primary to secondary, all the way to universities prefers teaching Python as primary programming languages.

## Local Literature

Computer science software development evolves rapidly; innovative learning platforms transform how instructors teach and how aspiring developers learn programming. Several other solutions integrate interactive web based on learning programming languages. However, these methods of studying are not applicable to everyone. This is where a game-based solution can complement the traditional way of learning (Horlador, et., 2024)

The rapid development of computer science education signifies that traditional methods of learning are being challenged. Conventional approaches often are lacking when it comes to engaging students or providing practical application of programming concepts (Horlador, et., 2024). Traditionally, a beginner developer sits inside a classroom, listening to an instructor as they explain related programming concepts. However, there are instances wherein the beginner developer cannot follow along with the instructor's sample codes (Horlador, et., 2024).

The current education system is outdated in the sense that it cannot keep up with the demands of a fast-paced and information-rich world outside of the classroom. In response to these challenges, innovative learning platforms can be applied to transform the way beginner developers usually learn (Horlador, et., 2024).

In today's rapidly evolving landscape, mastering the concepts of Information Technology is crucial for the students aspiring to enter the technology industry (Abat, et al., 2024). Software development is fundamental to the innovation and efficiency of the applications and solutions tailored to specific needs and demands (Abat, et al., 2024). This caused the current curriculum to include programming languages such as Lua and Python (Abat, et al., 2024).Despite the abundance of academic materials and tutorials regarding different types of programming languages, most of the students still face difficulties in learning the fundamentals of programming and how to apply them. By implementing interactive game-based learning, it allows students to learn using an alternative method of learning that is both fun and educational. (Abat et al., 2024).

**Foreign Studies**

According to Eltomi (2025), the use of Lua programming language allows the game developers to easily implement core gameplay mechanics such as character movements, physics, and collision detection. This study also identified the problems that game developers encountered such as no prior knowledge in Lua programming language, Game size and complexity, and bad game designs. In relation to the study, it explains the use of Lua programming language in game development. However, the proposed study lacks the idea of showing how the actual Lua language is used in game development.

In a study done by Verbovetsksvi (2025), The authors identified the basic requirements for a game application, such as relevance to educational objectives, incorporation of game mechanics, the ability to write and debug code in the application, individualized passing of the game, and the ability to report and rate learning outcomes of students. The paper contains some fragments of the game workspace and code of a gaming application.

Several studies compare the use of Python and Lua in game development, highlighting their distinct advantages. Python, with libraries like Pygame, is favored for its simplicity and extensive community support, making it ideal for rapid prototyping and educational game development (Oleksiuk et al., 2024). However, it tends to have slower performance compared to Lua. Lua is widely used in professional game engines due to its lightweight nature, fast execution, and seamless integration with C/C++ code, making it suitable for performance-critical game scripting (MoldStud, 2025).

Tyagi's study (2021) compares developing a 2D Flappy Bird game in both Python (Pygame) and Lua (LOVE engine), showing that while Python is easier for beginners, Lua provides better performance and embedding capabilities for commercial projects. Other research agrees that Lua excels in embedded scripting and resource-sensitive environments, whereas Python is often used for game prototyping, tooling, and educational purposes.

In conclusion, these studies demonstrate that the choice between Python and Lua depends on the project goals, with Lua being preferred for efficiency and embedding, and Python for ease of use and broad application.

## Local Studies

According to a study done in the University of Mindanao-PALMA Cluster Campus Philippines aimed to determine the effectiveness of game-based learning, based on the findings it can be affirmed that implementing game-based learning strategies is highly effective, resulting in a notable enhancement of learners' academic performance compared to conventional teaching methods. Moreover, the results encourage teachers to revisit their instructional materials and integrate game-based learning materials aligned with specific learning objectives. Additionally, the study provides substantial evidence that game-based learning significantly enhances academic performance compared to traditional instructional resources (Cezar, 2024).

According to Cezar (2022), a study was done to measure the effect of digital game-based learning. Educators and authors argued that video games are constructed on well-developed theories to engage players. The gathered data were interpreted and analyzed; the results revealed that the experimental group achieved higher scores in their academic performance test. The findings showed a significant difference in the performance for students engaging in digital game-based learning.

According to Justo et al. (2022), a study was done to analyze students' perspectives of game-based learning. The results have been proven the game-based learning has a more significant influence on retention and class interaction when this technique is used. As a result, the findings say that game-based learning aids information acquisition and boosts student engagement through increased class interaction.

## Foreign Systems

CodeCombat (2024) is a game-based learning platform that teaches languages such as Python through interactive, role-playing gameplay. Its custom-built coding engine and interpreter make it beginner friendly. The platform includes project-sharing features and teacher grading systems, making it usable in both personal and educational contexts. According to Ahlstrom (2025), the system not only simplifies coding, but they execute it in a method that emphasizes having fun. CodeCombat demonstrates how games can effectively serve as a medium for learning programming concepts.

OmegaCrafter (2025) is an open-world survival craft game set in an underdevelopment game world that is being attacked by a mysterious program. Grammi, the players' programmable sidekick can be programmed with various actions and tasks to help with mining, logging, moving materials, and crafting items (Preferred Networks, 2025)

The Farmer Was Replaced (2023) is a game for Python-like language to fully automate various farming tasks that would otherwise be time consuming. (Herzog, 2023). Unlike most programming games, it is not divided into distinct levels that players have to complete but instead features continuous progression. Programming is done in a simple language similar to Python. The game is designed to provide basic programming concepts to the player (Herzog, 2023).

## Local Systems

Philippine Coding Camp (2023) is a digital learning center that teaches Filipino students how to code through structured, hands-on programs. The system provides learning courses which teach Lua as one of the foundations for beginner and younger students while also introducing Python later as a part of the game. In relation to the study, the system highlights accessibility in programming education while sustaining engagement and interactivity that traditional digital courses may not fully achieve.

Logiscool (2025) is a digital learning platform that focuses on coding and digital literacy education. Its courses integrate popular platforms to provide engaging environments to enhance coding concepts. The system demonstrates the effectiveness of combining fun and interactivity with education.

Hello World (2024) is a game that teaches the fundamentals of Java and C#. Players apply their skills into gameplay where challenges are introduced. The game progresses based on what the players have learned so far, from introduction to more advanced topics. It is a practical application of game-based learning in the growing industry of programming.

## Synthesis

The reviewed foreign and local literature, studies, and systems collectively emphasize the growing importance of game-based learning in programming education. Foreign authors (Becker, 2022; Crudu, 2024; Viduka, 2021) highlighted how games foster immersion and focus, while also pointing to the industry relevance of programming languages such as Lua and Python. Local studies (Horlador et al., 2024; Abat et al., 2024) reinforced these ideas by showing how interactive, game-based methods can complement or even surpass traditional classroom instruction, especially in engaging students who struggle with conventional approaches. Moreover, both foreign and local systems (e.g., CodeCombat, 2024; Philippine Coding Camp, 2023) demonstrated practical applications of integrating coding into interactive platforms, providing evidence that game-based solutions are not only feasible but effective in real-world educational settings.

However, despite these advances, the reviewed literature also reveals a gap: while many systems teach Python or Lua through games, few explicitly focus on bridging the fundamentals of programming with practical coding immersion tailored for beginners in the Philippine context. This gap underscores the significance of the present study, which seeks to design a game-based solution that leverages the strengths of Lua and Python while addressing the challenges faced by novice developers in traditional learning environments. In the context of learning, both data gathered from foreign and local literature are that gaming tools have been used as a powerful supplementary tool for learning and can be applicable into learning complex topics such as concepts of programming (Becker, 2022; Horlador, et al., 2024).

The difference between the gathered information from foreign and local literatures are that in foreign literature, it has been found that games have been increasingly prominent in educational settings since when it comes to learning, students can be more focused and immersed (Becker, 2022). Local literature on the other hand has found that traditional methods are becoming outdated and are challenged by more innovative means of learning, especially considering the context of teaching complex topics in

Computer Science (Horlador, et al., 2024).

Foreign studies done by authors (Eltomi, 2025; MoldStud, 2025; Tiyagi, 2021) gathered that Lua and Python are applicable in implementing features much easier due to each of the advantages that the two languages have. Lua is a lightweight language and integrates with C and C++ code while Python is more beginner-friendly and applicable for prototyping and tooling.

Local studies proven by authors (Cezar, 2022; Justo, et al., 2022) found that gamified learning is an effective method of learning as it has been gathered that the group that was engaged in gamified learning scored better in comparison to the group that did not. Based on what the study has gathered from the local and foreign studies, the goal is to implement gamified learning into teaching students the concepts of Lua and Python as these two languages have been shown to be appealing for beginners to learn (Oleksiuk, et al., 2024; MoldStud, 2025).

The proposed game aims to be unique by implementing the methods of gamified learning into teaching the concepts of Lua and Python in such a way that it acts as a supplement alongside traditional learning methods. It acts to teach students in a more engaging way. It does so by applying concepts and ideas that the foreign and local literature and studies have gathered and combine them in such a way that it introduces a specialized method of teaching students various programming concepts. Lua and Python were chosen as the two programming languages to teach because of their simplicity and utility, making them appealing to teach.

# METHODOLOGY

## Waterfall Development Life Cycle

Waterfall methodology is a well-established project management workflow. Each process phase cascades downward sequentially through five stages, requirements, design, implementation, verification, and maintenance.

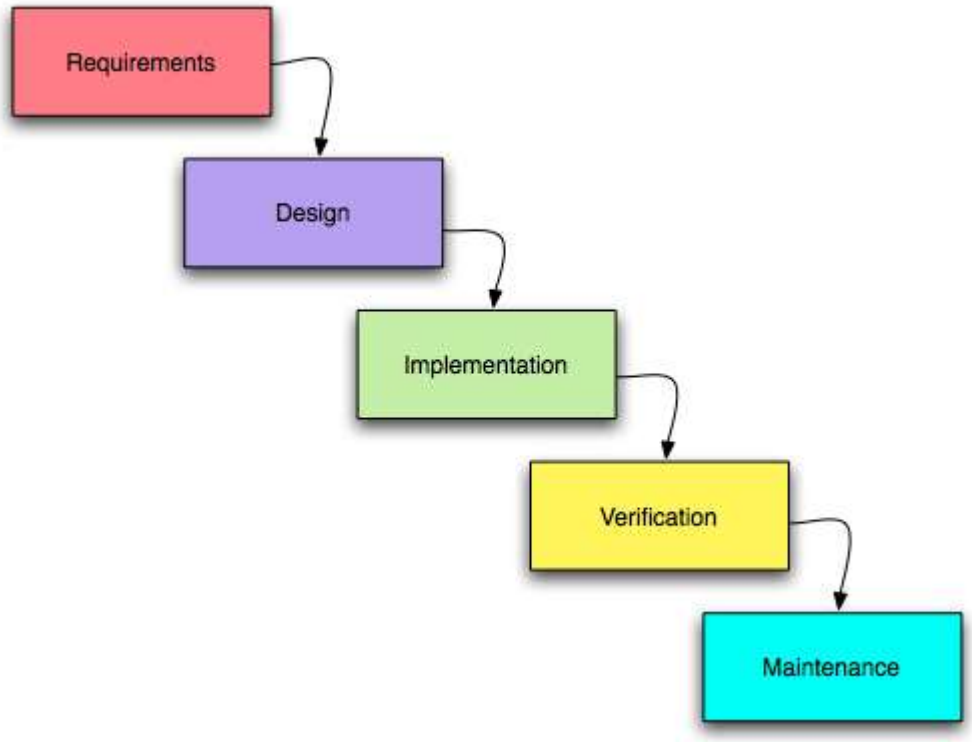



**Figure 1. Waterfall methodology**

**Requirements**

The requirement phase states what the system should do. At this stage the project of the scope is determined alongside the resources for the project, allocation of work for each member, timeline of the project, and the details of each stage.

**Design**

After gathering all the requirements, the developer develops solutions that meet the requirements. This phase focuses on the overall blueprint of the software including its user interface and architecture.

**Implementation**

Once the design is finalized and approved, the developers build from the specifications of design and collect research needed for the development.

**Verification**

The verification stage is dedicated to quality assurance and testing of the game alongside fixing the possible errors and bugs that may appear during development.

**Maintenance**

After the initial release, the developers address the additional bugs that may appear and continue to provide fixes that have not appeared during the initial verification stage. This includes fixing incompatibility issues provided within the limitations of the software and fixing bugs that occurs inside the game.

## Conceptual Framework

The study is guided by the input-process-output (IPO) model, a widely used framework for describing how a system transforms identified needs into a working solution and measurable results. The model is appropriate for SandEE because the game is both a software product and a learning intervention: its design must respond to the learners' needs, its development must follow a structured process, and its output must be evaluated in terms of both software quality and educational value.

The input component consists of students' learning needs, including their limited familiarity with Lua, interest in game-based learning, curriculum programming concepts, and platform requirements. The process component covers the Waterfall methodology and core game systems, including procedural generation, survival and crafting, mission progression, Lua and Python execution, and user interface development. The output component includes the SandEE game, its learning experiences, and user and expert evaluation results. Evaluation feedback guides improvements to the input and process for future iterations.

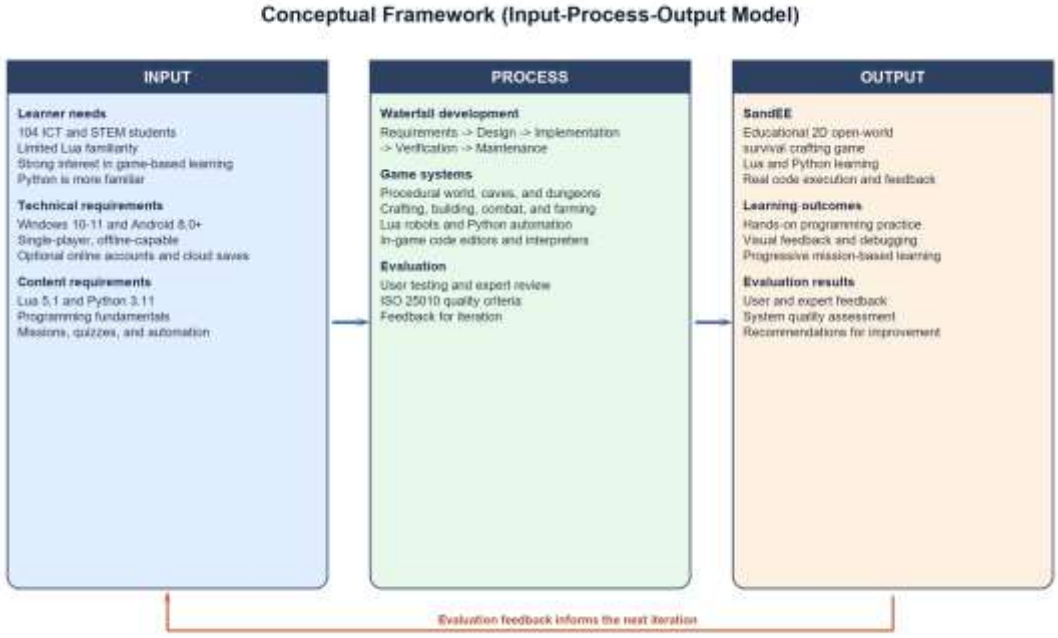



**Figure 2. Conceptual framework**

## System Design Specification

SandEE is implemented in Godot 4.6 using GDScript for the game logic and scene composition. The project follows a modular, scene-based architecture in which the overworld, cave interiors, dungeon interiors, player, user interface, and programming tools are separate scenes or scripts coordinated by a central game instance. Four autoloads support the runtime: CrashLogger records unexpected errors, Global stores persistent state and shared services, Supabase provides optional online authentication and data access, and CloudSync manages two-way cloud-save synchronization. The main entry point is the startup mode scene, which restores a session when available and leads to the main menu.

The game uses a client-side architecture. All core gameplay, world generation, saving, and code execution run on the player's device, which allows the game to remain playable offline. Online features are optional and are accessed only when the player signs in. The language runtime is implemented as a Rust GDExtension: Lua is executed through the mlua library, while Python source is parsed with ruff_python_parser and transpiled to GDScript before execution. This design keeps the game engine independent of the language implementations while still providing real parsing, execution, and error reporting.

**Functional Modules**

**Table 1. Functional modules and descriptions**

| Module | Description |
|---|---|
| World generation | Seeded terrain generation with plains, desert, snowy, and water areas; chunk-based structures, caves, and dungeon entrances. |
| Caves and dungeons | Cellular-automata caves, room-and-corridor pixie caves, and themed Lua/Python dungeons with knowledge and quiz altars. |
| Survival and crafting | Resource gathering, mining, smelting, crafting, building, farming, fishing, combat, and day/night survival. |
| Programming tools | In-game Lua and Python editors, syntax highlighting, code execution, error reporting, snippets, and the Programming Encyclopedia. |
| Automation | LUA robot stations that mine, chop, build, deposit, and fight; Algorithmica stations that automate furnaces, crafting stations, and color crafters. |
| Missions and progression | Mission tracker, blue and yellow stars, ranks, completion badges, dungeon rewards, and leaderboards. |
| Accounts and saves | Optional username/password accounts, local JSON saves, autosave, cloud saves, conflict handling, and star totals through Supabase. |
| User interface | Main menu, HUD, inventory, hotbar, crafting and smelting panels, minimap, pause menu, settings, and mobile touch controls. |

## System Architecture

The system is organized into five layers. The presentation layer contains the startup, menu, HUD, inventory, crafting, code-editor, and mobile-control interfaces. The game layer contains the overworld, caves, dungeons, player, combat, missions, and day/night cycle. The systems layer contains world generation, crafting and recipes, save/load, cloud sync, LUA robots, automation hosts, waypoints, and the minimap. The language runtime layer contains the Rust GDExtension, the Lua runtime, the Python parser and transpiler, error reporting, and the code sandbox. The data layer contains Godot resource files, local JSON saves, and the Supabase services used for accounts, cloud storage, and leaderboards.

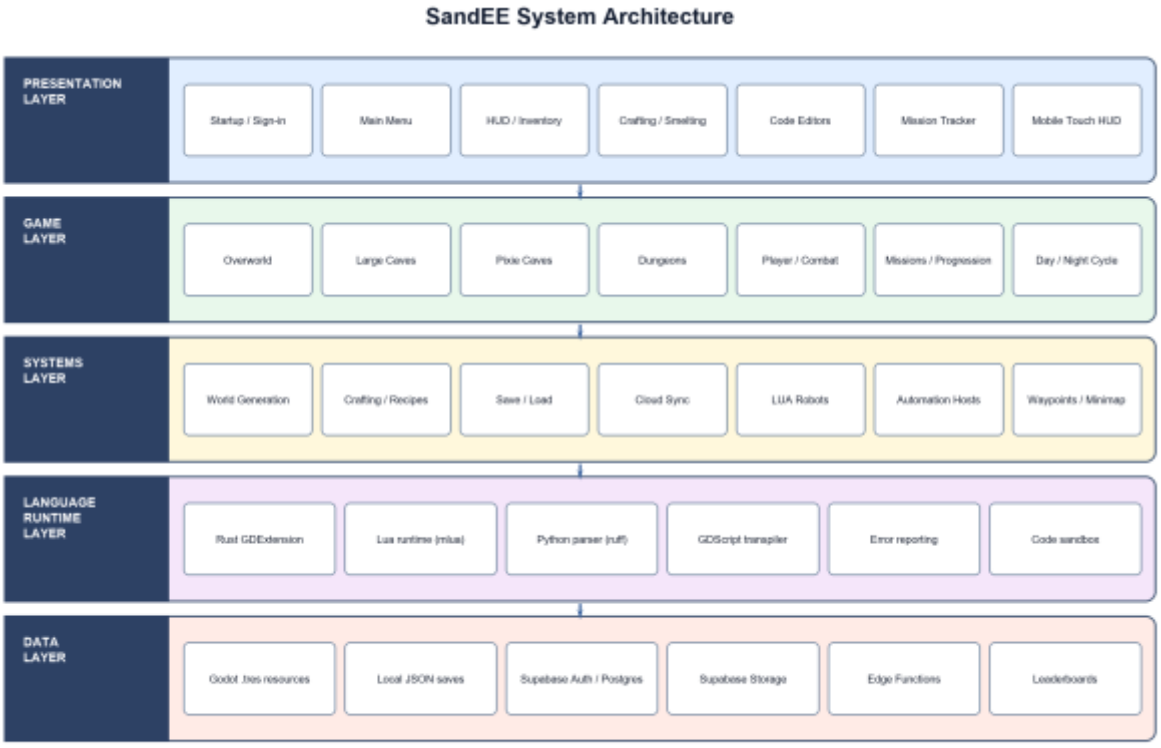



**Figure 3. System architecture**

## World Generation and Environment

The overworld is generated from a random seed and divided into 25 by 25-tile chunks. Terrain is classified into water, plain, desert, and snowy biomes, and each chunk is

populated with trees, grass, rocks, structures, mobs, and cave entrances according to seeded rules. Large caves use cellular-automata carving to produce organic caverns with stone and ore deposits. Pixie caves use room-and-corridor layouts with crystal ores and regenerating resources. Dungeons use a separate room-and-corridor generator that guarantees a knowledge room, a quiz room, an entrance room, and additional rooms for altars and chests. The world also includes a day/night cycle, ambient mob spawning, waypoints, and a minimap.

The world generator is deterministic: the same seed produces the same terrain, structures, cave entrances, and dungeon layouts. This is important for reproducibility, debugging, and fair play. The game also separates natural terrain from player-placed blocks, so the original world can be regenerated while player changes are restored from the save file.

**Programming and Automation Module**

The programming module is the educational core of SandEE. Players write Lua or Python in an in-game code editor that provides line numbers, syntax highlighting, snippets, and an output panel. Lua programs run through the embedded Lua runtime and can control LUA robots. Python programs are parsed and transpiled to GDScript before execution, and can control Algorithmica stations such as furnaces and crafting stations. The editors report syntax and runtime errors back to the player, and a successful run produces visible changes in the game world. This creates a direct link between code, feedback, and gameplay.

LUA robots can be renamed, given tools, assigned build materials, and switched into combat mode. They can chop trees, mine blocks, place blocks, deposit items, and fight nearby enemies. Algorithmica stations expose a Python API for loading inputs, adding fuel, smelting, crafting, and taking outputs. The Programming Encyclopedia and the Knowledge Altar provide reference material and unlockable explanations of programming concepts, while the Quiz Altar checks the player's understanding before a dungeon is cleared.

### Save, Account, and Cloud System

Each world is stored in a local JSON save file associated with an account and a save slot. The game autosaves periodically and at important state changes such as entering a cave or dungeon, and it also saves when the player quits. Saves are written atomically to avoid corruption, and a sidecar file records ownership so that saves cannot be mixed between accounts. When the player is signed in and online, CloudSync uploads the local save to Supabase Storage and records its metadata in the database. Cloud-only worlds can be downloaded into free local slots, and conflicts are resolved by revision, world identity, or manual selection. The online layer also supports profiles, star totals, and leaderboards, with row-level security enforced in the database.

The account and cloud features are optional. A player can create a local account and play offline, then switch to online mode later. This supports classrooms and devices with unreliable internet access while still allowing progress to be synchronized across devices.

### User Interface and Cross-Platform Layer

The user interface is designed for both Windows and Android. On desktop, the player uses the keyboard and mouse for movement, interaction, building, and combat. On mobile, a virtual joystick, contextual action buttons, and tap-to-interact controls replace the mouse and keyboard. The HUD includes the hotbar, health and stamina bars, mission tracker, minimap, inventory, crafting panel, code editors, and pause menu. All interface elements respect the global UI scale setting, which can be set to small, medium, or large. The game also uses a pixel-art visual style, responsive canvas scaling, and a consistent color palette across menus and in-game panels.

## Diagrams

The following diagrams describe the flow of data, the structure of the data, the functional hierarchy, the account authentication process, and the main gameplay loops of SandEE. The data flow diagrams show how the player, the game system, local saves,

and the optional cloud services exchange information. The entity relationship diagram shows the main data entities used by the save, mission, dungeon, and automation systems. The hierarchical chart shows the major modules of the game, and the flowcharts show the expected sequence of account access and player actions in the overworld and in dungeons.

### Data Flow Diagram Level 0

Figure 4 presents the context diagram of SandEE. The player interacts with the game by issuing commands, writing code, and making menu choices. The game returns visual feedback, error messages, game state, and progress information. The game reads and writes local save files, and it exchanges authentication, cloud-save, and leaderboard data with Supabase when online features are enabled.

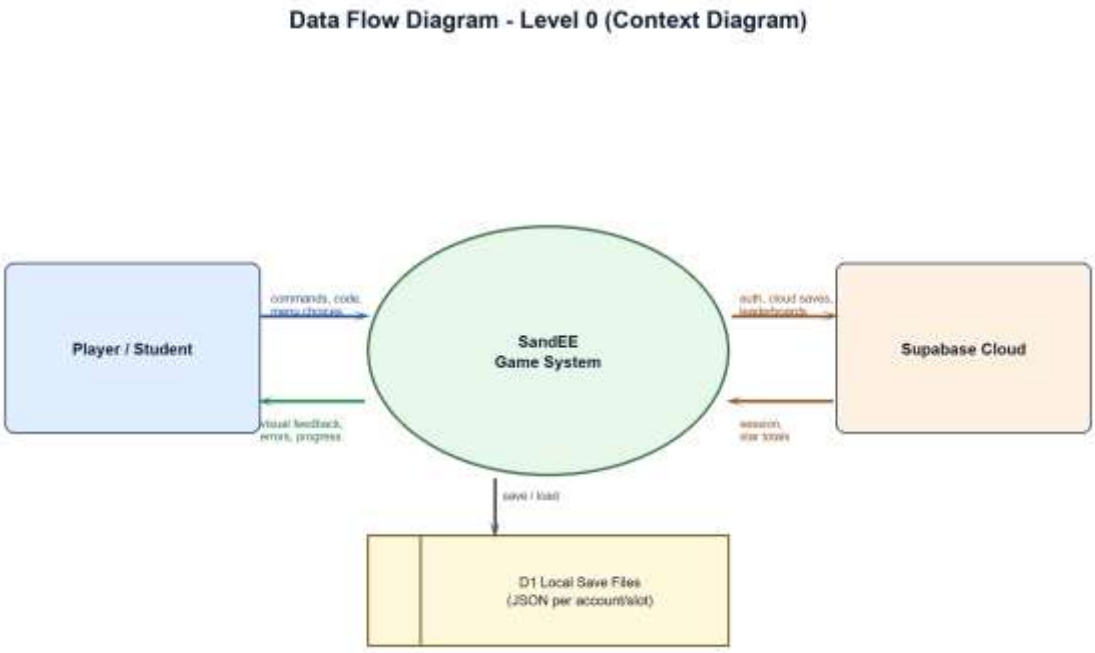



**Figure 4. Data flow diagram level 0**

### Data Flow Diagram Level 1

Figure 5 decomposes the system into six processes. Account and save management handles sign-in, local saves, and cloud synchronization. World exploration manages terrain, caves, structures, and mobs. Gathering, crafting, and building manage

resources, recipes, and placed blocks. Learning and code execution manage the editors, interpreters, and automation behavior. Mission and dungeon completion manage objectives, quizzes, rewards, and stars. Progress and leaderboard management maintain the player's profile, achievements, and ranking. The data stores hold the player profile, world save, mission progress, code programs, and game content.

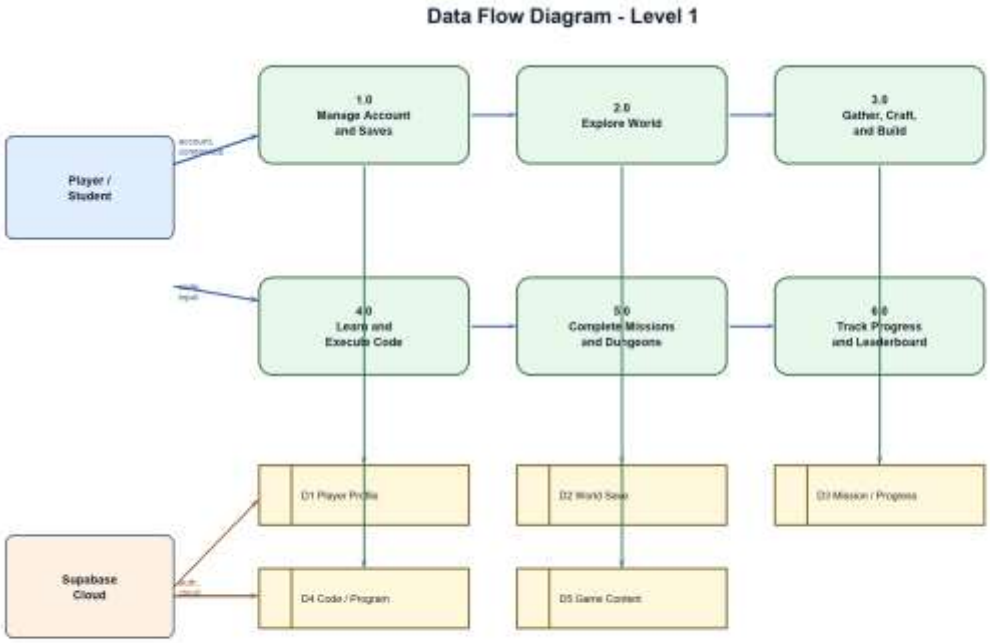



**Figure 5. Data flow diagram level 1**

## Entity Relationship Diagram

Figure 6 shows the main entities used by the latest version of SandEE. A player can own multiple world saves, mission-progress records, cloud saves, bestiary entries, and encyclopedia unlocks. Each world save contains inventory slots, waypoints, LUA robots, and automation stations. Missions generate dungeons, and each dungeon contains rooms with knowledge or quiz altars. Recipes reference the items they consume and produce, while LUA robots and automation stations reference the code programs that control them. These relationships support the save, mission, dungeon, programming, and online systems.

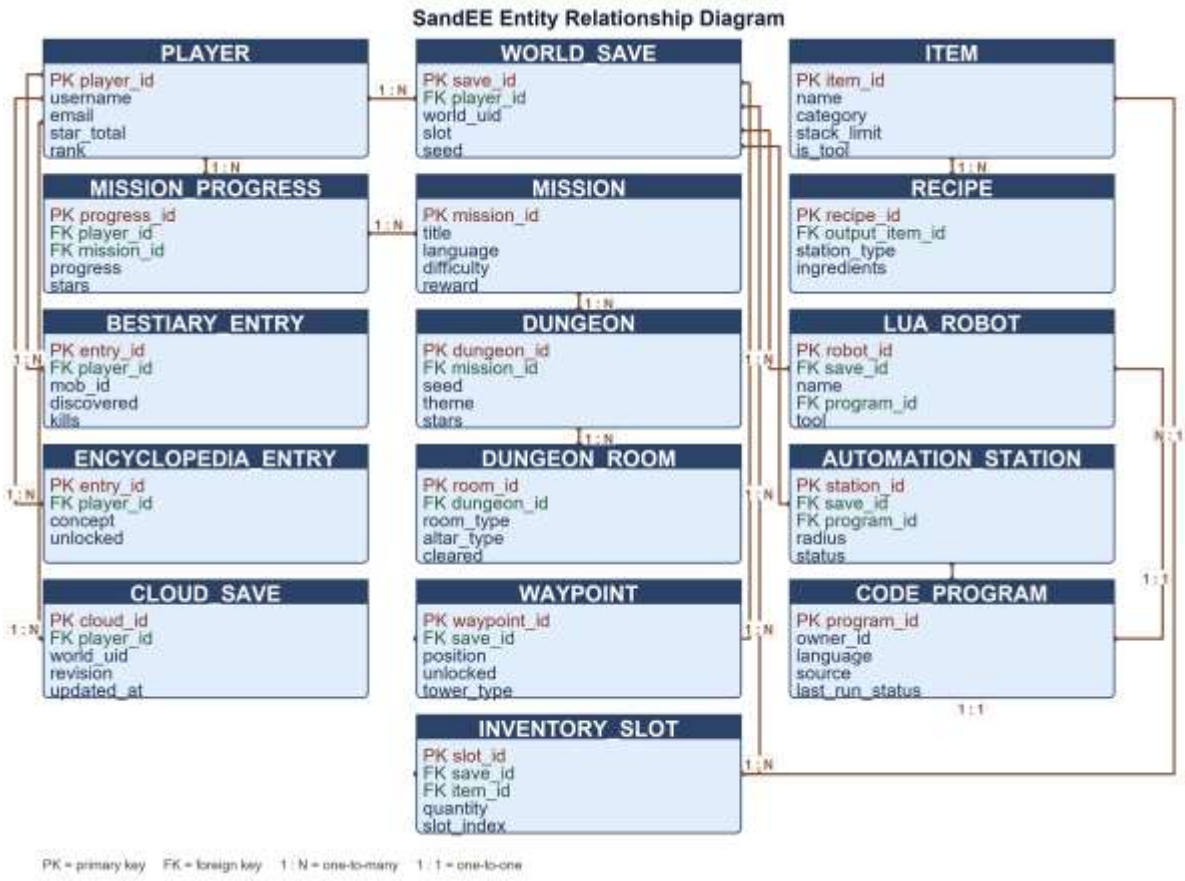



**Figure 6. Entity relationship diagram**

## Hierarchical Chart

Figure 7 presents the functional hierarchy of SandEE. The main menu and account module provides access to worlds, leaderboards, cloud saves, and settings. The overworld module contains biomes, trees, water, caves, structures, and mobs. The cave module contains large caves, pixie caves, ores, and mining. The dungeon module contains the Lua and Python dungeons, knowledge and quiz altars, and the miniboss encounter. The programming module contains the Lua and Python editors, LUA robots, Algorithmica, and the encyclopedia. The interface module contains the inventory, crafting, mission tracker, minimap, and mobile HUD. The online module contains authentication, cloud saves, leaderboards, and profiles.

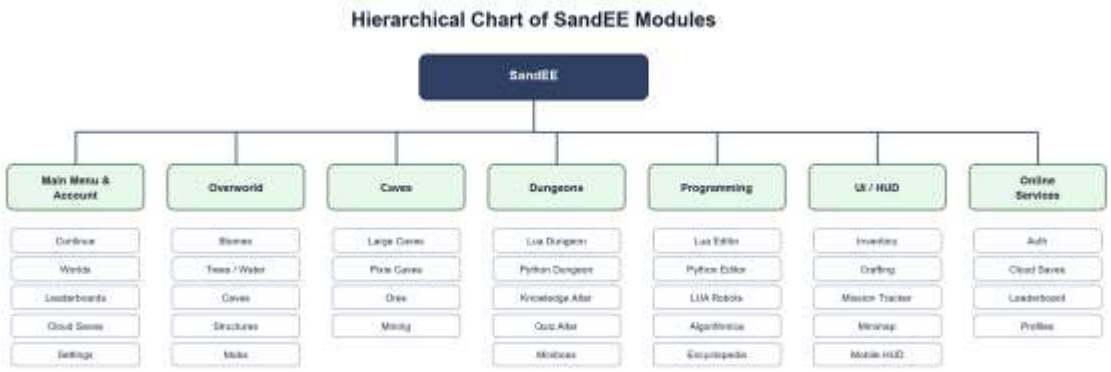



**Figure 7. Hierarchical chart**

### Account Authentication Flowchart

Figure 8 presents the account authentication flow of SandEE. The startup screen first checks internet access and the availability of the Supabase backend. If the service is unavailable, the game offers a verified offline copy when one exists; otherwise, it shows a connection error and allows the player to retry. When the service is available, the game restores and validates a stored session. A valid session proceeds directly to profile refresh and cloud synchronization, while an invalid or missing session opens the login and registration form. Successful sign-in or registration refreshes the player profile, begins an account-scoped session, initializes cloud-save metadata, and synchronizes local and cloud worlds before opening the main menu. Signing out clears the account context and returns the player to the login form.

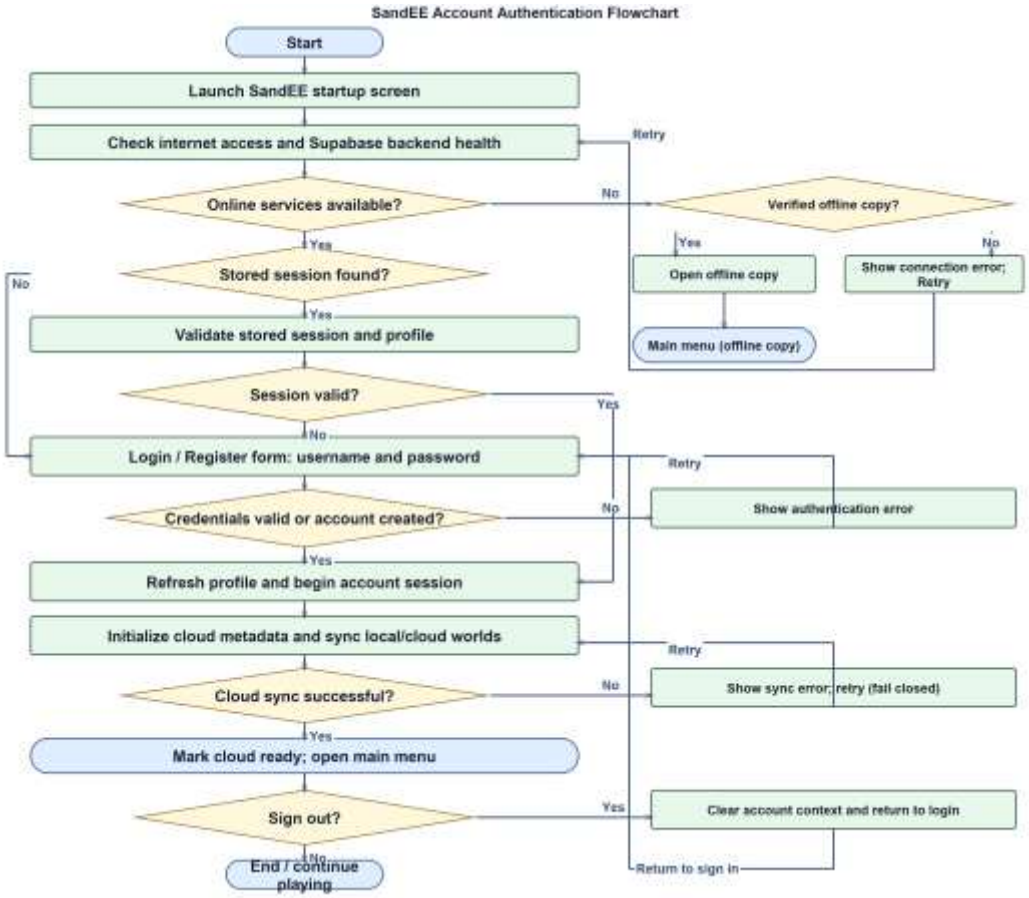



**Figure 8. Account authentication flowchart**

### Overworld Gameplay Flowchart

Figure 9 shows the gameplay loop of SandEE after authentication. From the main menu, the player loads or creates a world and follows the tutorial and guided objectives. The player explores the overworld, gathers resources, crafts items, builds structures, manages the inventory, and programs LUA robots, Algorithmica stations, or Python programs. Missions, day/night encounters, caves, waypoints, and dungeons provide progression. Entering a dungeon leads to the Knowledge Altar, quiz and code challenges, miniboss encounter, and star rewards before returning to the overworld. The loop ends with autosave and cloud synchronization.

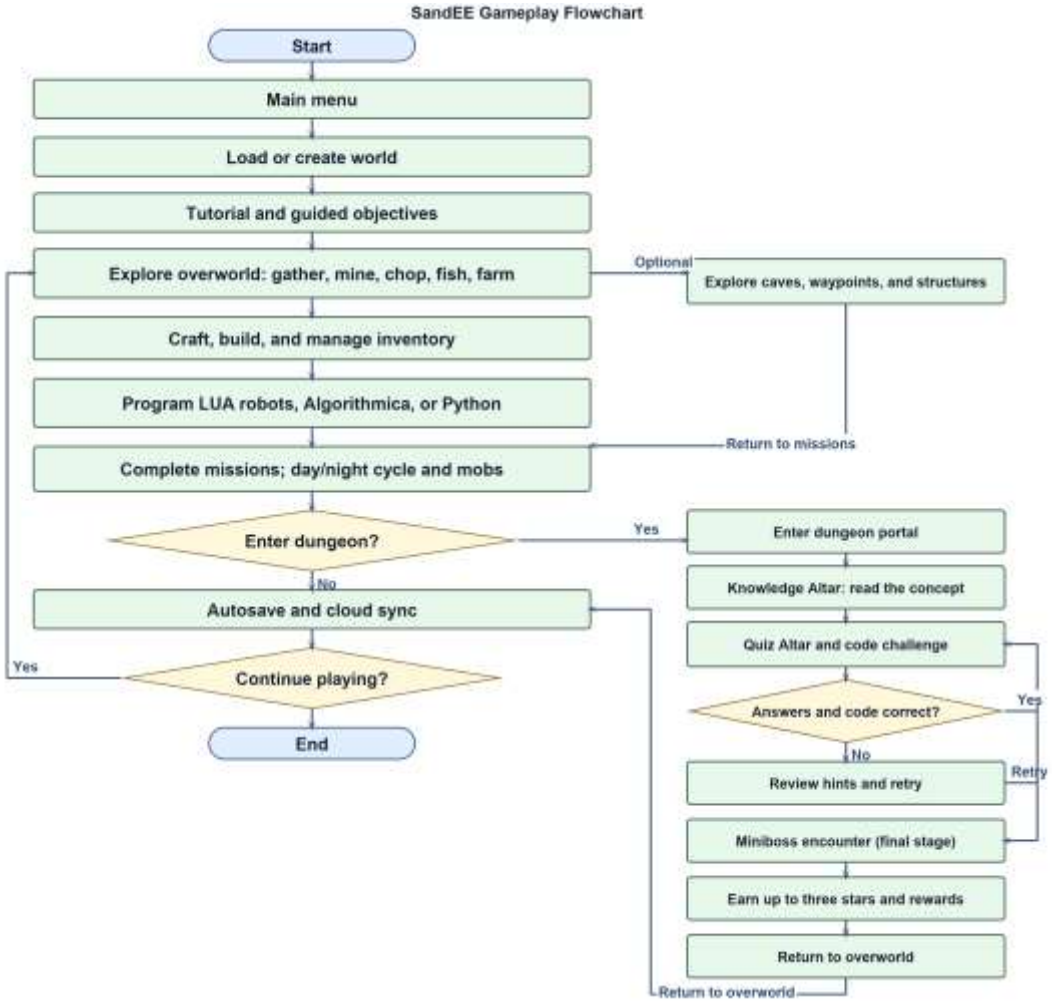



**Figure 9. Updated gameplay flowchart**

**Dungeon Gameplay Flowchart**

Figure 10 shows the flow inside a dungeon. The player enters the dungeon, explores its rooms, reads the Knowledge Altar, answers the Quiz Altar questions, and writes and runs Lua or Python code. Correct answers and working code earn up to three stars. The final stage includes a miniboss encounter. Completing the dungeon rewards the player and returns them to the overworld.

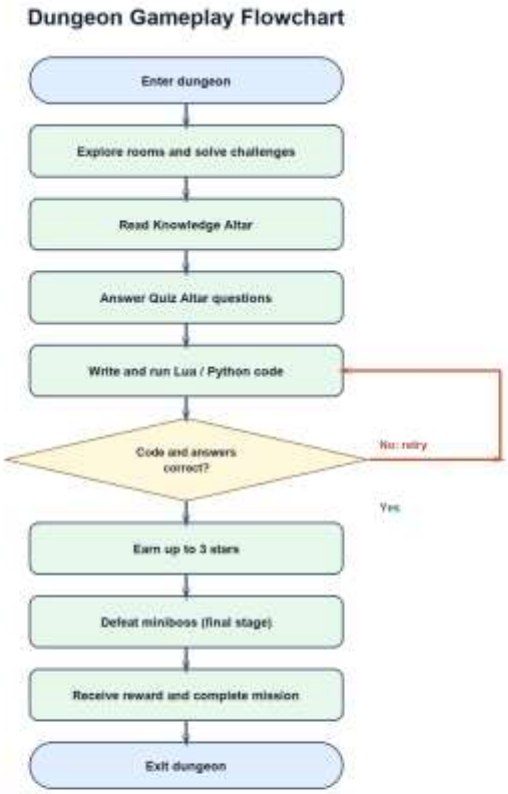



**Figure 10. Dungeon gameplay flowchart**

The diagrams show that SandEE is not a single linear program but a set of interacting systems. The player's actions in one system, such as mining an ore or writing a program, affect the state of other systems, such as crafting, missions, and saves. This interdependence is what makes the game a coherent open-world experience rather than a collection of separate mini-games.

**Hardware/Software**

**Table 2. Hardware**

| Devices | Specifications |
|---|---|
| Desktop | CPU: AMD Ryzen 5 5600G<br>Motherboard: MSI PRO B550M-P GEN3<br>GPU: Radeon Vega 7 Integrated Graphics<br>RAM: 16 GB<br>Operating System: CachyOS (Arch Linux) |
| Laptop | CPU: i5-12500h<br>Motherboard:AN515-58-50YE<br>GPU: RTX 3050M<br>Ram: 8x2 DDR4 2400mhz sodimm<br>Operating System: Windows 10 Pro |

**Table 3. Software**

| Software | Purpose |
|---|---|
| Godot Engine | Game Engine |
| GDScript | Game Logic |
| Supabase | Game Database |
| Aseprite | Art Editing Software |
| Rust | Performance critical implementation |
| Lua | Main Scripting Language |
| Python | Sub Scripting Language |
| VS Codium | Chosen IDE. |
| GitHub | Version Control Software, Logger, File Storage |

## Calendar of Activities

During the first week of September, the developers began working on the initial draft of the study's introduction and finalized the research title. This phase focuses on identifying the study's purpose, significance, and scope while aligning the objectives with the project's intended outcome. The week concluded with the team reviewing the overall direction of the research to ensure that it addressed relevant academic and industry needs.

In the second week of September, the team revised the introduction to improve clarity and alignment with the research objectives. They also began drafting Chapter I, which detailed the background of the problem, statement of the problem, and significance of the study. The group collaborated closely to ensure that the content presented a strong foundation for the succeeding chapters.

The third week of September was dedicated to additional revisions and refinements of Chapter I. The developers reviewed the document to ensure consistency, coherence, and adherence to academic standards. Feedback from peers and advisers was considered to enhance the quality and flow of the content, preparing the chapter for final review.

During the fourth week of September, the developers conducted the survey to gather data from the target respondents. The collected information served as the basis for understanding user preferences and needs. In the same week, the team began drafting Chapter II, which included the review of related literature, studies, and systems relevant to the proposed project.

The first week of October was devoted to extensive research on literature, existing studies, and software systems related to game-based learning. The developers consolidated and analyzed this information to strengthen Chapter II. By the end of the week, Chapter II was completed and prepared for initial revision.

In the second week of October, the proponents focused on drafting the hardware and software selection section. This included identifying the tools, technologies, and development environments necessary for system implementation. The week also involved outlining the technical specifications and requirements for the proposed application.

During the third week of October, the team concentrated on revising and improving Chapter III, which presented the project's methodology. The developers refined the research design, data collection methods, and system development framework to ensure accuracy and alignment with research objectives. In the fourth week of October, the proponents carried out final revisions and grammar checks across all chapters to ensure consistency and correctness. The team also completed the initial design of the software user interface, marking the transition from the documentation phase to the beginning of system development.

## Budgetary Estimate

### Table 4. Budgetary Estimate

| Quantity | Specifics | Approximate Cost |
|---|---|---|
| 1 | Acer Nitro 5 AN515-50YE | PHP 49,000 |
| 1 | Ryzen 5 5600G processor | PHP 22,295 |
| | Total | PHP 71,295 |

# RESULTS AND DISCUSSIONS

## Overview of the results

This chapter presents the results of the IT student and IT expert evaluation of SandEE. The IT student evaluation assessed the game's functional suitability, performance efficiency, and usability based on selected ISO 25010 criteria. A total of 70 IT students participated. The IT expert evaluation assessed the game's functional suitability, performance efficiency, compatibility, usability, reliability, security, maintainability, and portability.

The results are presented using tables. Frequency and percentage distributions are used for the student evaluation, while weighted means and standard deviations are used to summarize the responses. The interpretation of the weighted means follows the four-point ISO 25010 quality scale presented in Table 5.

## System Implementation

Figure 11 shows actual screenshots of SandEE during gameplay. The screenshots document the Lua Ancient Rune Sanctum, a dungeon exploration area, and the Python Stonegate Foundry. These images provide evidence that the game was implemented and that the visual style, dungeon themes, and player interface are consistent with the design described in the methodology.



**Figure 11. SandEE gameplay screenshots**

## Data Gathering Procedure

The student evaluation was conducted after the game was developed. The IT students were given a short demonstration of the game's main features, including the main menu, world exploration, crafting, building, the Lua and Python editors, LUA robots, Algorithmica stations, and the dungeon challenges. They were then asked to evaluate the game using a four-point ISO 25010 quality scale questionnaire. The evaluation involved 70 IT students and three IT experts, namely Harold James Galve, Miguel Mercado, and Ken Ryan Calvez. The IT experts were selected based on their background in software development, programming education, or game development.

For the IT students, the instrument covered functional suitability, performance efficiency, and usability. For the IT experts, it covered functional suitability, performance efficiency, compatibility, usability, reliability, security, maintainability, and portability. Each criterion was represented by several statements, and the respondents rated each statement from 1 (Poorly Functional/Efficient/Usable) to 4 (Very Functional/Efficient/Usable), depending on the ISO 25010 criterion. The responses were tallied, and the weighted mean and standard deviation were computed for each criterion.

## Statistical Treatment of Data

The data were analyzed using frequency, percentage, weighted mean, and standard deviation. Frequency refers to the number of respondents who selected a particular response, while percentage is the frequency divided by the total number of respondents multiplied by 100. The weighted mean is computed by multiplying each response value by its frequency, summing the products, and dividing the sum by the total number of responses. The standard deviation measures the spread of the responses around the mean; a small standard deviation indicates that the respondents' ratings were consistent.

The weighted mean is interpreted using the four-point ISO 25010 quality scale in Table 5. A mean within 3.26–4.00 is interpreted as Very Functional, Very Efficient, or Very Usable; 2.51–3.25 as Functional, Efficient, or Usable; 1.76–2.50 as Moderately Functional, Moderately Efficient, or Moderately Usable; and 1.00–1.75 as Poorly Functional, Poorly Efficient, or Poorly Usable.

**Table 5. Mean range interpretation legend**

| Mean Range | Interpretation |
|---|---|
| 3.26 – 4.00 | Very High / Very Functional / Very Efficient / Very Usable |
| 2.51 – 3.25 | High / Functional / Efficient / Usable |
| 1.76 – 2.50 | Low / Moderately Functional / Moderately Efficient / Moderately Usable |
| 1.00 – 1.75 | Very Low / Poorly Functional / Poorly Efficient / Poorly Usable |

**User Feedback**

This section presents the results of the IT student evaluation of SandEE. A total of 70 IT students participated in the evaluation and rated the game using a four-point ISO 25010 quality scale. The responses were tallied, and the weighted mean and standard deviation were computed for each item. The results are presented per indicator, followed by a summary by criterion.

**Table 6. Functional completeness - Degree to which the set of functions covers all the specified tasks and user objectives**

| Metric | Poorly Functional | Moderately Functional | Functional | Very Functional |
|---|---|---|---|---|
| Average Frequency (%) | 0 | 0 | 37.14% | 62.86% |
| Rank | 1 | 2 | 3 | 4 |

The results show that 62.86% of the IT students rated SandEE as Very Functional for functional completeness, while 37.14% rated it Functional. No respondents rated it Moderately Functional or Poorly Functional. This indicates that most IT students perceive the system as functional and that it covers all the specified tasks and user objectives.

**Table 7. Functional correctness - Degree to which a product or system provides the correct results with the needed degree of precision**

| Metric | Poorly Functional | Moderately Functional | Functional | Very Functional |
|---|---|---|---|---|
| Average Frequency (%) | 0 | 0 | 34.29% | 65.71% |
| Rank | 1 | 2 | 3 | 4 |

The results show that 65.71% of the IT students rated SandEE as Very Functional for functional correctness, while 34.29% rated it Functional. No respondents rated it Moderately Functional or Poorly Functional. This indicates that most IT students perceive the system as functional and that it provides correct results with the needed degree of precision.

**Table 8. Functional appropriateness - Degree to which the functions facilitate the accomplishment of specified tasks and objectives**

| Metric | Poorly Functional | Moderately Functional | Functional | Very Functional |
|---|---|---|---|---|
| Average Frequency (%) | 0 | 0 | 40.00% | 60.00% |
| Rank | 1 | 2 | 3 | 4 |

The results show that 60.00% of the IT students rated SandEE as Very Functional for functional appropriateness, while 40.00% rated it Functional. No respondents rated it Moderately Functional or Poorly Functional. This indicates that most IT students perceive the system as functional and that its functions facilitate the accomplishment of specified tasks and objectives.

**Table 9. Time behaviour - Degree to which the response and processing times and throughput rates of system, when performing functions, meet requirements**

| Metric | Poorly Efficient | Moderately Efficient | Efficient | Very Efficient |
|---|---|---|---|---|
| Average Frequency (%) | 0 | 0 | 45.71% | 54.29% |
| Rank | 1 | 2 | 3 | 4 |

The results show that 54.29% of the IT students rated SandEE as Very Efficient for time behaviour, while 45.71% rated it Efficient. No respondents rated it Moderately Efficient or Poorly Efficient. This indicates that most IT students perceive the system as efficient and that its response and processing times and throughput rates meet the requirements.

**Table 10. Resource utilization - Degree to which the amounts and types of resources used by a system, when performing its functions, meet requirements**

| Metric | Poorly Efficient | Moderately Efficient | Efficient | Very Efficient |
|---|---|---|---|---|
| Average Frequency (%) | 0 | 0 | 48.57% | 51.43% |
| Rank | 1 | 2 | 3 | 4 |

The results show that 51.43% of the IT students rated SandEE as Very Efficient for resource utilization, while 48.57% rated it Efficient. No respondents rated it Moderately Efficient or Poorly Efficient. This indicates that most IT students perceive the system as efficient and that it uses appropriate amounts and types of resources when performing its functions.

**Table 11. Capacity - Degree to which the maximum limits of a product or system parameter meet requirements**

| Metric | Poorly Efficient | Moderately Efficient | Efficient | Very Efficient |
|---|---|---|---|---|
| Average Frequency (%) | 0 | 0 | 42.86% | 57.14% |
| Rank | 1 | 2 | 3 | 4 |

The results show that 57.14% of the IT students rated SandEE as Very Efficient for capacity, while 42.86% rated it Efficient. No respondents rated it Moderately Efficient or Poorly Efficient. This indicates that most IT students perceive the system as efficient and that its maximum limits meet the requirements.

**Table 12. Appropriateness recognizability - Degree to which users can recognize whether a system is appropriate for their needs**

| Metric | Poorly Usable | Moderately Usable | Usable | Very Usable |
|---|---|---|---|---|
| Average Frequency (%) | 0 | 0 | 35.71% | 64.29% |
| Rank | 1 | 2 | 3 | 4 |

The results show that 64.29% of the IT students rated SandEE as Very Usable for appropriateness recognizability, while 35.71% rated it Usable. No respondents rated it Moderately Usable or Poorly Usable. This indicates that most IT students perceive the system as usable and that users can recognize whether SandEE is appropriate for their needs.

**Table 13. Learnability - Degree to which a system can be used by specified users to achieve specified goals of learning to use the product or system with effectiveness, efficiency, freedom from risk and satisfaction in a specified context of use**

| Metric | Poorly Usable | Moderately Usable | Usable | Very Usable |
|---|---|---|---|---|
| Average Frequency (%) | 0 | 0 | 32.86% | 67.14% |
| Rank | 1 | 2 | 3 | 4 |

The results show that 67.14% of the IT students rated SandEE as Very Usable for learnability, while 32.86% rated it Usable. No respondents rated it Moderately Usable or Poorly Usable. This indicates that most IT students perceive the system as usable and that it can be learned effectively, efficiently, and with satisfaction.

**Table 14. Operability - Degree to which a system has attributes that make it easy to operate and control**

| Metric | Poorly Usable | Moderately Usable | Usable | Very Usable |
|---|---|---|---|---|
| Average Frequency (%) | 0 | 0 | 38.57% | 61.43% |
| Rank | 1 | 2 | 3 | 4 |

The results show that 61.43% of the IT students rated SandEE as Very Usable for operability, while 38.57% rated it Usable. No respondents rated it Moderately Usable or Poorly Usable. This indicates that most IT students perceive the system as usable and that it is easy to operate and control.

**Table 15. User error protection - Degree to which a system protects users against making errors**

| Metric | Poorly Usable | Moderately Usable | Usable | Very Usable |
|---|---|---|---|---|
| Average Frequency (%) | 0 | 0 | 44.29% | 55.71% |
| Rank | 1 | 2 | 3 | 4 |

The results show that 55.71% of the IT students rated SandEE as Very Usable for user error protection, while 44.29% rated it Usable. No respondents rated it Moderately Usable or Poorly Usable. This indicates that most IT students perceive the system as usable and that it protects users against making errors.

**Table 16. User interface aesthetics - Degree to which a user interface enables pleasing and satisfying interaction for the user**

| Metric | Poorly Usable | Moderately Usable | Usable | Very Usable |
|---|---|---|---|---|
| Average Frequency (%) | 0 | 0 | 31.43% | 68.57% |
| Rank | 1 | 2 | 3 | 4 |

The results show that 68.57% of the IT students rated SandEE as Very Usable for user interface aesthetics, while 31.43% rated it Usable. No respondents rated it Moderately Usable or Poorly Usable. This indicates that most IT students perceive the system as usable and that its user interface enables pleasing and satisfying interaction.

**Table 17. Accessibility - Degree to which a system can be used by people with the widest range of characteristics and capabilities to achieve a specified goal in a specified context of use**

| Metric | Poorly Usable | Moderately Usable | Usable | Very Usable |
|---|---|---|---|---|
| Average Frequency (%) | 0 | 0 | 41.43% | 58.57% |
| Rank | 1 | 2 | 3 | 4 |

The results show that 58.57% of the IT students rated SandEE as Very Usable for accessibility, while 41.43% rated it Usable. No respondents rated it Moderately Usable or Poorly Usable. This indicates that most IT students perceive the system as usable and that it can be used by people with a wide range of characteristics and capabilities.

**Summary of IT Student Evaluation Results**

**Table 18. Summary of IT student evaluation results**

| Criterion | Weighted Mean | Standard Deviation | Interpretation |
|---|---|---|---|
| Functional Suitability | 3.63 | 0.48 | Very Functional |
| Performance Efficiency | 3.54 | 0.50 | Very Efficient |
| Usability | 3.63 | 0.48 | Very Usable |
| Overall | 3.61 | 0.49 | Very High |

The summary shows that functional suitability obtained a weighted mean of 3.63 (SD = 0.48), interpreted as Very Functional; performance efficiency obtained 3.54 (SD = 0.50), interpreted as Very Efficient; and usability obtained 3.63 (SD = 0.48), interpreted as Very Usable. All three criteria fall within the 3.26–4.00 range. The overall weighted mean of 3.61 (SD = 0.49) is interpreted as Very High, indicating that the IT students rated SandEE favorably across the evaluated ISO 25010 characteristics.

**Expert Feedback**

The IT experts who evaluated the game were Harold James Galve, Miguel Mercado, and Ken Ryan Calvez. They provided qualitative feedback on its design, functionality, and educational value. They commended the modular structure of the system, the use of real Lua and Python execution, and the consistency of the pixel-art interface. They also noted that the game provides clear progression from simple exploration to programming-based automation, which makes it suitable for beginner learners.

The experts recommended [insert expert recommendation 1], [insert expert recommendation 2], and [insert expert recommendation 3]. These recommendations were considered in the final revisions of the game and are reflected in the recommendations section of this study.

# CONCLUSION

## Summary

This study developed and evaluated SandEE, an educational multi-platform 2D open-world survival crafting game designed to supplement the learning of Lua and Python among Grade 11 and 12 ICT and STEM students of Yverdon de Pestalozzi School Inc. The game combines procedural world generation, survival and crafting, mission-based progression, themed Lua and Python dungeons, knowledge and quiz altars, a programming encyclopedia, and embedded language interpreters that allow player-written programs to control LUA robots and Algorithmica stations. It runs on Windows and Android, supports offline play, and offers optional online accounts, cloud saves, and leaderboards. The study used the Waterfall model and evaluated the completed system through an ISO 25010-based survey of 70 IT students and a qualitative review by three IT experts.

The IT student evaluation produced an overall weighted mean of 3.61 (SD = 0.49), interpreted as Very High. Functional suitability obtained 3.63 (SD = 0.48), performance efficiency 3.54 (SD = 0.50), and usability 3.63 (SD = 0.48), all within the Very Functional, Very Efficient, and Very Usable ranges. Every indicator received majority ratings of Very Functional, Very Efficient, or Very Usable, and no respondent rated any indicator as Poorly or Moderately. The IT experts likewise affirmed the modular structure of the system, the authenticity of its Lua and Python execution, the consistency of its pixel-art interface, and the clarity of its progression from exploration to programming-based automation. These results show that SandEE met its functional requirements and was received favorably in terms of performance and usability.

## Conclusions

SandEE was found to be Very Functional, with a weighted mean of 3.63 (SD = 0.48). Functional completeness was rated Very Functional by 62.86% of the IT students, functional correctness by 65.71%, and functional appropriateness by 60.00%; the remaining respondents rated these indicators Functional, and none rated them Moderately Functional or Poorly Functional. These results confirm that the game covers the specified tasks and user objectives, produces correct results with the needed precision, and provides functions that directly facilitate the intended learning activities. The combination of exploration, resource gathering, crafting, building, missions, dungeons, and programming challenges gives players a coherent set of activities that support the study's objectives rather than isolated features.

Performance efficiency obtained 3.54 (SD = 0.50), interpreted as Very Efficient. Time behaviour was rated Very Efficient by 54.29% of the respondents, resource utilization by 51.43%, and capacity by 57.14%, with the rest rating these indicators Efficient. Although all performance indicators were positive, the relatively lower rating for resource utilization shows that memory use, asset loading, and performance on less powerful devices remain the most practical areas for improvement. Usability obtained 3.63 (SD = 0.48), interpreted as Very Usable. Appropriateness recognizability was rated Very Usable by 64.29%, learnability by 67.14%, operability by 61.43%, user error protection by 55.71%, user interface aesthetics by 68.57%, and accessibility by 58.57%. These findings indicate that players can recognize the purpose of the game, learn its controls and programming workflow, operate its interface, and recover from errors, while the comparatively lower scores for error protection and accessibility identify where further usability work is most needed.

The expert assessment supported these findings. The three IT experts commended the modular structure of the system, the use of genuine Lua and Python execution rather than simulated outputs, and the consistency of the pixel-art interface. They also observed that the game presents a clear progression from simple exploration and resource gathering to mission-based challenges and programming-driven automation, which makes it appropriate for beginner learners. Their feedback confirmed that the functional, instructional, and visual components of SandEE work together as an integrated educational game and that the system performs its intended functions accurately and consistently during evaluation.

Taken together, the student and expert evaluations show that SandEE achieved its development goals. The overall weighted mean of 3.61 (SD = 0.49) is interpreted as Very High, and the narrow spread of responses indicates that the positive ratings were consistent across the respondents. The results establish that the game is functional, efficient, and usable, and that it can serve as a supplementary tool for programming education by giving students a contextual environment in which to practice Lua and Python. The findings also show that the game does not replace classroom instruction or the formal curriculum; instead, it reinforces both by allowing students to apply programming concepts, observe the results of their code, and learn from errors in a controlled and engaging setting. The remaining limitations concern performance on constrained hardware and specific usability details rather than the core functions of the system.

## Recommendations

Based on the results and conclusions of this study, several recommendations are offered to improve the functionality, performance, usability, and long-term value of SandEE. These recommendations focus on the areas that received the lowest ratings and on the features that would broaden the game's usefulness in actual classroom settings.

Future developers should prioritize performance optimization because performance efficiency received the lowest criterion mean (3.54), with resource utilization receiving the lowest indicator rating (51.43% Very Efficient). Memory consumption should be reduced through more efficient asset loading, texture and audio compression, and unloading of unused world regions. World generation and save operations should be profiled and optimized to avoid delays during exploration, and the Android build should be tested on low-end devices to identify frame-rate drops, excessive battery use, and storage growth. Providing graphics quality settings, view-distance controls, and a battery-saving mode would allow users to adjust the game to the capabilities of their devices without sacrificing the core experience.

The mission and dungeon content should be expanded to cover more programming concepts, including functions, conditional statements, loops, data structures, file handling, and object-oriented programming. Additional Lua and Python examples, hints, and guided tutorials should be integrated into the missions and knowledge altars so that beginners can recover from mistakes without becoming frustrated. A teacher or facilitator mode is also recommended, allowing instructors to assign missions, monitor student progress, review code submissions, and export learning reports. This feature would make SandEE easier to integrate into formal lessons and would give teachers evidence of student practice and improvement.

Usability improvements should focus on user error protection and accessibility, which received the lowest usability indicator ratings. Error messages in the Lua and Python editors should be rewritten in clearer language, with the location of the error, a short explanation, and a suggested correction. Confirmation prompts for destructive actions, an undo function for building and crafting, and automatic recovery of unsaved work would reduce the consequences of mistakes. Accessibility should be improved through remappable controls, colorblind-friendly palettes, adjustable text sizes, subtitles or visual cues for important audio, and additional language options. A more detailed onboarding sequence and an in-game help panel would also help new players understand the controls, the programming workflow, and the relationship between code and in-game actions.

Finally, the development team should continue optimizing the Android build and explore releases for iOS and web browsers to reach more students and schools. Cooperative or shared-world features could allow students to collaborate on programming tasks and compare their solutions, while maintaining offline play for classrooms with limited connectivity. A longitudinal study should measure learning outcomes over a longer period and compare game-based practice with traditional programming exercises. Consistent documentation, version control, bug tracking, and regular updates will support the maintainability and long-term scalability of the system. Continued feedback from students, teachers, and IT experts should guide future revisions so that SandEE remains responsive to the evolving needs of its users.

## REFERENCES


Pohl, M., Rester, M., Judmaier, P. (2009). Interactive Game Based Learning: Advantages and Disadvantages. In: Stephanidis, C. (eds) Universal Access in Human-Computer Interaction. Applications and Services. UAHCI 2009. Lecture Notes in Computer Science, vol 5616. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-642-02713-0_10

D. Zhao, C. H. Muntean, A. E. Chis, G. Rozinaj and G. -M. Muntean, "Game-Based Learning: Enhancing Student Experience, Knowledge Gain, and Usability in Higher Education Programming Courses," in IEEE Transactions on Education, vol. 65, no. 4, pp. 502-513, Nov. 2022, doi: 10.1109/TE.2021.3136914.

Lampropoulos, G. (2023). Educational benefits of digital game-based learning: K-12 teachers' perspectives and attitudes. Advances in Mobile Learning Educational Research, 3(2), 805-817. https://doi.org/10.25082/AMLER.2023.02.008

Becker, B. A., Denny, P., Finnie-Ansley, J., Luxton-Reilly, A., Prather, J., & Santos, E. A. (2023). Programming is hard—or at least it used to be: Educational opportunities and challenges of AI code generation. In Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1 (SIGCSE 2023) (pp. 500–506). Association for Computing Machinery. https://doi.org/10.1145/3545945.3569759

Brumbaugh, Z., & Leithner, M. (2023). The Lua programming language. In Programming languages: Concepts and implementations (pp. 45–60). Springer. https://doi.org/10.1007/979-8-8688-0060-3_3

Crudu, V., & MoldStud Research Team. (2024, September 27). Are Lua developers in high demand in the job market? MoldStud. https://moldstud.com/articles/p-are-lua-developers-in-high-demand-in-the-job-market

Ierusalimschy, R., Figueiredo, L. H. de, & Celes, W. (2025). The evolution of Lua, continued. Journal of Computer Languages, 83, 101326. https://doi.org/10.1016/j.cola.2025.101326

Lin, H.-C. K., Tseng, C.-H., & Chen, N.-S. (2025). Enhancing programming education: The impact of AI-based pedagogical agents on student self-efficacy, engagement, and learning outcomes. Educational Technology & Society, 28(2), 279–294. https://www.jstor.org/stable/48827987

Python, W. (2021). Python: Python releases for Windows. Python Software Foundation. https://www.python.org/downloads/windows/

Viduka, D., Kraguljac, V., & Ličina, B. (2021). A comparative analysis of the benefits of Python and Java for beginners. Quaestus, (19), 318–327.

Abat, J., et al. (2024). [Complete title of study]. [Journal/Publisher]. [Details to be verified].

Ahlstrom, [Initial]. (2025). [Title of article or resource]. [Publisher/URL]. [Details to be verified].

Becker, B. A., Denny, P., Finnie-Ansley, J., Luxton-Reilly, A., Prather, J., & Santos, E. A. (2022). Programming is hard - or at least it used to be: Educational opportunities and challenges of AI code generation. arXiv. https://arxiv.org/abs/2212.01020

Cezar, [Initial]. (2022). [Title of study]. [Journal/Publisher]. [Details to be verified].

Cezar, [Initial]. (2024). [Title of study]. [Journal/Publisher]. [Details to be verified].

CodeCombat. (2024). CodeCombat: Learn to code by playing a game. https://codecombat.com

CodinGame. (2025). CodinGame: Programming games and competitions. https://www.codingame.com

Eltomi, [Initial]. (2025). [Title of study]. [Journal/Publisher]. [Details to be verified].

Herzog, [Initial]. (2023). The Farmer Was Replaced. [Steam/Publisher/URL]. [Details to be verified].

Horlador, [Initial]., et al. (2024). [Title of study]. [Journal/Publisher]. [Details to be verified].

Justo, [Initial]., et al. (2022). [Title of study]. [Journal/Publisher]. [Details to be verified].

Logiscool. (2025). Logiscool: Coding and digital literacy. https://www.logiscool.com

MoldStud. (2025). [Title of article]. MoldStud. [URL]. [Details to be verified].

Oleksiuk, [Initial]., et al. (2024). [Title of study]. [Journal/Publisher]. [Details to be verified].

OmegaCrafter. (2025). OmegaCrafter. [Steam/Publisher/URL]. [Details to be verified].

Philippine Coding Camp. (2023). Philippine Coding Camp. [URL]. [Details to be verified].

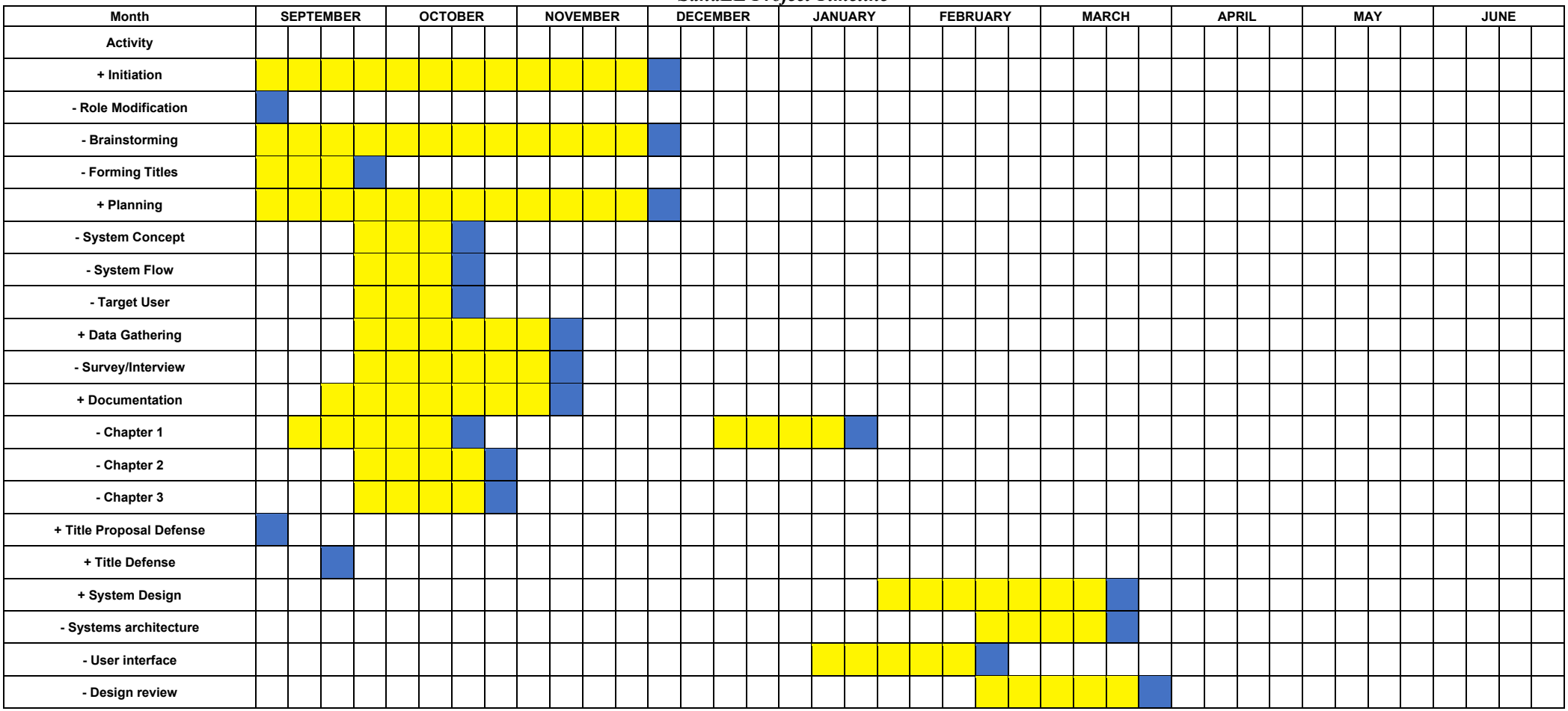
Tyagi, [Initial]. (2021). [Title of study]. [Journal/Publisher]. [Details to be verified].

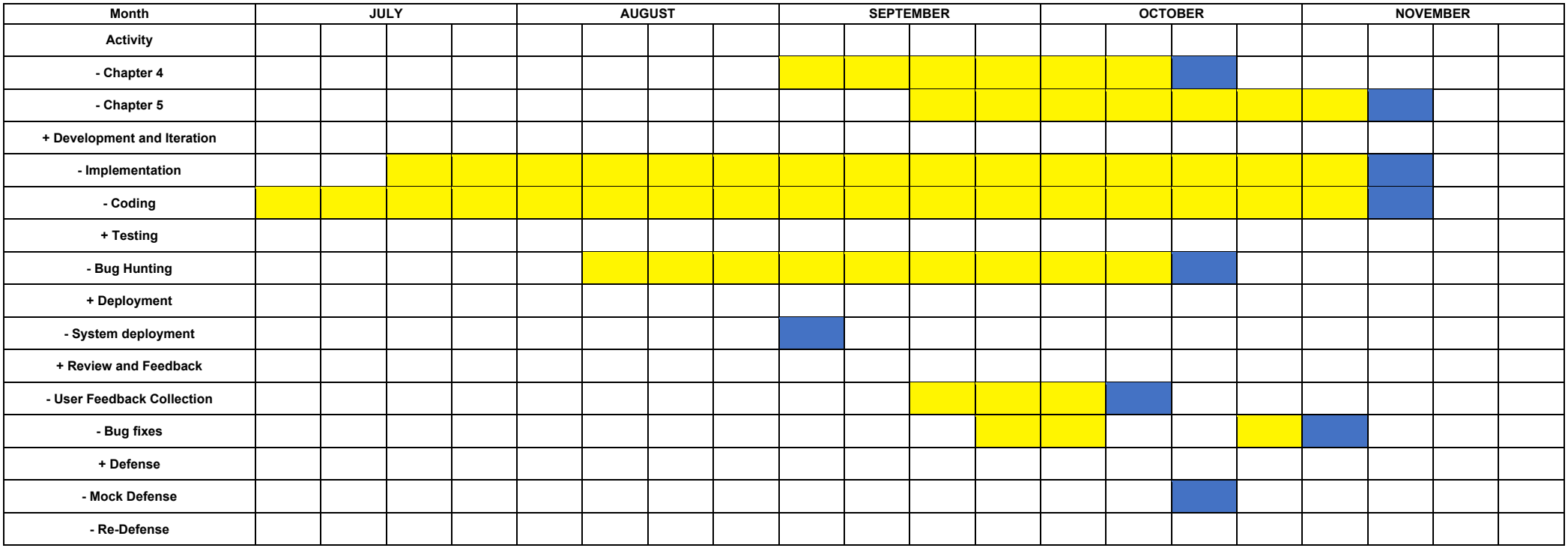
Verbovetskyi, [Initial]. (2025). [Title of study]. [Journal/Publisher]. [Details to be verified].

Hello World. (2024). Hello World. [Steam/Publisher/URL]. [Details to be verified].

# APPENDIX A. GANTT CHART

## *SandEE Project Timeline*




Month
JULY
AUGUST
SEPTEMBER
OCTOBER
NOVEMBER
Activity
- Chapter 4
- Chapter 5
+ Development and Iteration
- Implementation
- Coding
+ Testing
- Bug Hunting
+ Deployment
- System deployment
+ Review and Feedback
- User Feedback Collection
- Bug fixes
+ Defense
- Mock Defense
- Re-Defense

## APPENDIX B. ACTUAL THESIS EXPENSES

Table 19 presents the actual expenses incurred during the development and documentation of the study. The approximate costs were based on the budgetary estimate, while the actual costs reflect the final expenditures.

**Table 19. Actual thesis expenses**

| Quantity | Specifics | Approximate Cost | Actual Cost |
|---|---|---|---|
| 1 | Acer Nitro 5 AN515-50YE | PHP 49, 000 | 49, 000 |
| 1 | Ryzen 5 5600G processor | PHP 22, 295 | 22, 295 |
| | Total | 71295 | 71295 |
| | | | |

**Prepared by:**

**Glenn Marco B. Accad**
**Adrian V. Gallano**
**Jefferson G. Mamauag**
**Jose Mari Christian V. Tayag**

**Noted by:**

**[Name of Parent/Guardian]** **[Name of Parent/Guardian]**

**Approved by:**

**Norvin Shaq F. Lintag** **Enoch James C. Tayaguna**

# APPENDIX C. USER'S MANUAL

This user's manual provides the basic instructions for installing, launching, and playing SandEE. It covers the system requirements, controls, main menu, gameplay basics, programming tools, dungeons, saving, and troubleshooting.

**System Requirements**

• Windows 10 or Windows 11 (64-bit)

• Android 8.0 or later

• At least 4 GB of RAM

• At least 500 MB of free storage

• Keyboard and mouse for Windows, or touch screen for Android

• Optional internet connection for online accounts and cloud saves

**Installation**

1. Download the SandEE installer or Android package from the official distribution page.
2. On Windows, run the installer and follow the on-screen instructions. On Android, allow installation from the chosen source if prompted, then install the package.
3. Launch SandEE from the Start menu or the Android app drawer.

**Starting the Game**

When the game starts, the player can sign in with an existing account, create a new account, or choose offline mode. Online mode enables cloud saves and leaderboards. Offline mode allows the player to play without an internet connection, and the player can switch to online mode later from the main menu.

**Main Menu**

The main menu provides Continue, Worlds, Leaderboards, Cloud Saves, Account, Settings, and Quit. Continue loads the most recent save. Worlds opens the save-slot menu where the player can create, rename, delete, or load a world. Settings allows the player to adjust the volume, fullscreen mode, UI scale, and other options.

## Controls

**Table 20. SandEE controls**

| Action | Windows | Android |
|---|---|---|
| Move | WASD / arrow keys | Virtual joystick |
| Look / aim / point and click | Mouse | Tap |
| Interact / mine / place / attack | Left click | Contextual action buttons |
| Build mode | Ctrl | BUILD button |
| Sprint | Shift | SPRINT button |
| Inventory | E | Inventory button |
| Hotbar | 1-9 | Tap hotbar slots |
| Pause / back | Esc | Pause / back button |
| Toggle control guide | F1 | On-screen controls |

## Gameplay Basics

The player begins in a procedurally generated world. Trees, rocks, grass, water, and ores can be interacted with to gather resources. The hotbar shows the currently selected item, and the inventory stores additional items. Resources can be crafted into tools, weapons, food, and building blocks. The player can place blocks in build mode to create shelters, bridges, and other structures. The day and night cycle affects enemy spawning and difficulty.

## Programming Tools

The Lua editor is used to program LUA robots. The player can write code, run it, stop it, and view the output. The Python editor is used to program Algorithmica stations such as furnaces and crafting stations. Both editors provide syntax highlighting, snippets, and error messages. The Programming Encyclopedia contains explanations of programming concepts, and the Knowledge Altar provides additional references inside dungeons.

## Dungeons and Missions

Dungeons are special challenge areas that test the player's programming knowledge. Each dungeon contains a Knowledge Altar, a Quiz Altar, and a final challenge. The Quiz Altar asks questions about Lua or Python, and the player must answer them correctly to earn stars. The final stage includes a miniboss. Missions guide the player through the game and reward stars that determine the player's rank.

### Saving and Cloud Saves

The game saves automatically at regular intervals and at important state changes. The player can also save manually from the pause menu. When online, the save is uploaded to the cloud. If a conflict occurs, the game compares the local and cloud revisions and asks the player to choose which save to keep. Cloud saves can be managed from the main menu.

### Troubleshooting

- If the game does not start, verify that the device meets the system requirements and that the latest version is installed.
- If the Lua or Python editor reports that the language runtime is unavailable, reinstall the game or check that the extension files are present.
- If cloud saves do not sync, check the internet connection and sign in again.
- If the game runs slowly on Android, lower the UI scale and close other applications.

## APPENDIX D. SURVEY

The following ISO 25010 questionnaires were used in the study. The first form was administered to the IT student respondents, and the second form was administered to the IT experts.

*Questionnaire for IT Student Respondents*

Name: _____ Age: _____

**FUNCTIONAL SUITABILITY**

| Questions | 1 Poorly Functional | 2 Moderately Functional | 3 Functional | 4 Very Functional |
|---|---|---|---|---|
| Functional completeness - Degree to which the set of functions covers all the specified tasks and user objectives. | | | | |
| Functional correctness - Degree to which a product or system provides the correct results with the needed degree of precision. | | | | |
| Functional appropriateness - Degree to which the functions facilitate the accomplishment of specified tasks and objectives. | | | | |

**PERFORMANCE EFFICIENCY**

| Questions | 1 Poorly Efficient | 2 Moderately Efficient | 3 Efficient | 4 Very Efficient |
|---|---|---|---|---|
| Time behaviour - Degree to which the response and processing times and throughput rates of a system, when performing its functions, meet requirements. | | | | |
| Resource utilization - Degree to which the amounts and types of resources used by a system, when performing its functions, meet requirements. | | | | |
| Capacity - Degree to which the maximum limits of a product or system parameter meet requirements. | | | | |

**USABILITY**

| Questions | 1 Poorly Usable | 2 Moderately Usable | 3 Usable | 4 Very Usable |
|---|---|---|---|---|
| Appropriateness recognizability - Degree to which users can recognize whether a system is appropriate for their needs. | | | | |
| Learnability - Degree to which a system can be used by specified users to achieve specified goals of learning to use the product or system with effectiveness, efficiency, freedom from risk and satisfaction in a specified context of use. | | | | |
| Operability - Degree to which a system has attributes that make it easy to operate and control. | | | | |
| User error protection - Degree to which a system protects users against making errors. | | | | |
| User interface aesthetics - Degree to which a user interface enables pleasing and satisfying interaction for the user. | | | | |
| Accessibility - Degree to which a system can be used by people with the widest range of characteristics and capabilities to achieve a specified goal in a specified context of use. | | | | |

**PART 4: FEEDBACK AND SUGGESTIONS**

Kindly identify its strong point/s to be enhanced and its weak points that need to be addressed for improvements.

_____

_____

_____

Do you have any thoughts on other features or functions that can be added to SandEE? Kindly share any suggestions with us so we can better improve it.

_____
_____
_____

Respondent's Signature: _____ Date of Evaluation: _____

*Questionnaire for IT Experts*

**PART 1: RESPONDENT'S PROFILE**

Name: _____

Age: _____

Affiliated Institution:

_____

Position/Designation:

_____

**PART 2: SOFTWARE QUALITY EVALUATION**

Directions: Please evaluate the software quality of SandEE based on the eight main characteristics of the ISO 25010 Software Product Quality Standards. Choose the appropriate box using the four-point scale with its corresponding verbal descriptions (4 as the highest and 1 as the lowest).

**FUNCTIONAL SUITABILITY**

| Questions | 1 Poorly Functional | 2 Moderately Functional | 3 Functional | 4 Very Functional |
|---|---|---|---|---|
| Functional completeness - Degree to which the set of functions covers all the specified tasks and user objectives. | | | | |
| Functional correctness - Degree to which a product or system provides the correct results with the needed degree of precision. | | | | |
| Functional appropriateness - Degree to which the functions facilitate the accomplishment of specified tasks and objectives. | | | | |

**PERFORMANCE EFFICIENCY**

| Questions | 1 Poorly Efficient | 2 Moderately Efficient | 3 Efficient | 4 Very Efficient |
|---|---|---|---|---|
| Time behaviour - Degree to which the response and processing times and throughput rates of a product or system, when performing its functions, meet requirements. | | | | |
| Resource utilization - Degree to which the amounts and types of resources used by the system, when performing its functions, meet requirements. | | | | |

| | | | | |
|---|---|---|---|---|
| Capacity - Degree to which the maximum limits of the system parameter meet requirements. | | | | |

**COMPATIBILITY**

| Questions | 1 Poorly Compatible | 2 Moderately Compatible | 3 Compatible | 4 Very Compatible |
|---|---|---|---|---|
| Co-existence - Degree to which a system can perform its required functions efficiently while sharing a common environment and resources with other products, without detrimental impact on any other product. | | | | |
| Interoperability - Degree to which two or more systems, products or components can exchange information and use the information that has been exchanged. | | | | |

**USABILITY**

| Questions | 1 Poorly Usable | 2 Moderately Usable | 3 Usable | 4 Very Usable |
|---|---|---|---|---|
| Appropriateness recognizability - Degree to which users can recognize whether a system is appropriate for their needs. | | | | |
| Learnability - Degree to which a system can be used by specified users to achieve specified goals of learning to use the system with effectiveness, efficiency, freedom from risk and satisfaction in a specified context of use. | | | | |
| Operability - Degree to which a system has attributes that make it easy to operate and control. | | | | |
| User error protection - Degree to which a system protects users against making errors. | | | | |
| User interface aesthetics - Degree to which a user interface enables pleasing and satisfying interaction for the user. | | | | |
| Accessibility - Degree to which a system can be used by people with the widest range of characteristics and capabilities to achieve a specified goal in a specified context of use. | | | | |

**RELIABILITY**

| Questions | 1<br>Poorly Reliable | 2<br>Moderately Reliable | 3<br>Reliable | 4<br>Very Reliable |
|---|---|---|---|---|
| Maturity - Degree to which a system or component meets needs for reliability under normal operation. | | | | |
| Availability - Degree to which a system or component is operational and accessible when required for use. | | | | |
| Fault tolerance - Degree to which a system or component operates as intended despite the presence of hardware or software faults. | | | | |
| Recoverability - Degree to which, in the event of an interruption or a failure, a system can recover the data directly affected and re-establish the desired state of the system. | | | | |

**SECURITY**

| Questions | 1<br>Poorly Secure | 2<br>Moderately Secure | 3<br>Secure | 4<br>Very Secure |
|---|---|---|---|---|
| Confidentiality - Degree to which a system ensures that data are accessible only to those authorized to have access. | | | | |
| Integrity - Degree to which a system component prevents unauthorized access to, or modification of, computer programs or data. | | | | |
| Non-repudiation - Degree to which actions or events can be proven to have taken place so that the events or actions cannot be repudiated later. | | | | |
| Accountability - Degree to which the actions of an entity can be traced uniquely to the entity. | | | | |
| Authenticity - Degree to which the identity of a subject or resource can be proved to be the one claimed. | | | | |

**MAINTAINABILITY**

| Questions | 1<br>Poorly Maintainable | 2<br>Moderately Maintainable | 3<br>Maintainable | 4<br>Very Maintainable |
|---|---|---|---|---|
| Modularity - Degree to which a system or computer program is composed of discrete components such that a change to one component has minimal impact on other components. | | | | |
| Reusability - Degree to which an asset can be used in more than one system, or in building other assets. | | | | |
| Analysability - Degree of effectiveness and efficiency with which it is possible to assess the impact on a product or system of an intended change to one or more of its parts, or to diagnose a product for deficiencies or causes of failures, or to identify parts to be modified. | | | | |
| Modifiability - Degree to which a product or system can be effectively and efficiently modified without introducing defects or degrading existing product quality. | | | | |
| Testability - Degree of effectiveness and efficiency with which test criteria can be established for a system, product or component and tests can be performed to determine whether those criteria have been met. | | | | |

**PORTABILITY**

| Questions | 1 Poorly Portable | 2 Moderately Portable | 3 Portable | 4 Very Portable |
|---|---|---|---|---|
| Adaptability - Degree to which a product or system can effectively and efficiently be adapted for different or evolving hardware, software or other operational or usage environments. | | | | |
| Installability - Degree of effectiveness and efficiency with which a product or system can be successfully installed and/or uninstalled in a specified environment. | | | | |
| Replaceability - Degree to which a product can replace another specified software product for the same purpose in the same environment. | | | | |

**PART 4: FEEDBACK AND SUGGESTIONS**

Kindly identify its strong point/s to be enhanced and its weak points that need to be addressed for improvements.

_____
_____
_____

Do you have any thoughts on other features or functions that can be added to SandEE? Kindly share any suggestions with us so we can better improve it.

_____
_____
_____

Respondent's Signature: _____ Date of Evaluation: _____

- If progress is lost, check the save slot and the cloud-save menu before creating a new world.

## APPENDIX E. CURRICULUM VITAE OF RESEARCHERS

Curriculum Vitae of

# MAMAUAG, JEFFERSON G.

**<Block 66 Lot 24 Nuvista Subdivision Barangay Santo Cristo City of San Jose Del Monte Bulacan>**
**mamauag.248577@sjdelmonte.sti.edu.ph**
**0931096819**

## EDUCATIONAL BACKGROUND

| Level | Inclusive Dates | Name of school/ Institution |
|---|---|---|
| Tertiary | August 2023 | STI College San Jose Del Monte |
| Vocational/Technical | | |
| High School | May 2016-2023 | Yverdon De Pestalozzi School |
| Elementary | June 2010-2016 | Placido Del Mundo Elementary School |

## PROFESSIONAL OR VOLUNTEER EXPERIENCE

| Inclusive Dates | Nature of Experience Job Title | Name and Address of Company or Organization |
|---|---|---|

## AFFILIATIONS

| Inclusive Dates | Name of Organization | Position |
|---|---|---|
| July 2021-2023 | Code Camp Society | Member |
| August 2023-2024 | Intellitech | Member |
| August 2024-2025 | Intellitech | CS 2nd Year representative |
| August 2025-2026 | Intellitech | Member |

## SKILLS

| SKILLS | Level of Competency | Date Acquired |
|---|---|---|
| MS Word | Intermediate | September 2016 |
| Java Programming | Intermediate | December 2023 |
| Python Programming | Intermediate | April 2025 |

## TRAININGS, SEMINARS OR WORKSHOP ATTENDED

| Inclusive Dates | Title of Training, Seminar or Workshop |
|---|---|
| August 2023 | National Service Training Program |
| February 2024 | GitGud |

**Curriculum Vitae of**

# Tayag, Jose Mari Christian V.

Block 34, Lot 5, Saint Therese Phase, Deca Homes Subdivision, Barangay Loma de Gato, Marilao, Bulacan
tayag.334839@sjdelmonte.sti.edu.ph
+639166248648

## EDUCATIONAL BACKGROUND

| Level | Inclusive Dates | Name of school/ Institution |
|---|---|---|
| **Tertiary** | **August 2023** | **STI College San Jose Del Monte** |
| **Vocational/Technical** | | |
| **High School** | **May 2014-2020** | **Caloocan National Science and Technology Highschool** |
| **Elementary** | **June 2008-2014** | **ST. Anne Power Academy-Marilao, inc.** |

## AFFILIATIONS

| Inclusive Dates | Name of Organization | Position |
|---|---|---|
| August 2023-2024 | Intellitech | Member |
| August 2024-2025 | Intellitech | Member |
| August 2025-2026 | Intellitech | Member |

## SKILLS

| SKILLS | Level of Competency | Date Acquired |
|---|---|---|
| Ux/Ui Design | Intermediate | September 2020 |
| Python Programming | Intermediate | March 2018 |
| Rust Programming | Intermediate | December 2024 |

## TRAININGS, SEMINARS OR WORKSHOP ATTENDED

| Inclusive Dates | Title of Training, Seminar or Workshop |
|---|---|
| **August 2023** | **National Service Training Program** |
| **February 2024** | **GitGud** |
| | **CodeFest** |

**Curriculum Vitae of**

# Accad, Glenn Marco B.

**Block 32, Lot 17 Phase 6C Towerville, Barangay Gaya - Gaya, Towerville, San Jose Del Monte Bulacan**
**Accad.355761@sjdelmonte.sti.edu.ph**
**+639981847987**

## EDUCATIONAL BACKGROUND

| Level | Inclusive Dates | Name of school/ Institution |
|---|---|---|
| Tertiary | August 2023 | STI College San Jose Del Monte |
| Vocational/Technical | | |
| High School | May 2017-2023 | Graceville National High school |
| Elementary | June 2011-2017 | Marangal Elementary School |

## PROFESSIONAL OR VOLUNTEER EXPERIENCE

| Inclusive Dates | Nature of Experience Job Title | Name and Address of Company or Organization |
|---|---|---|
| February 2023 – May 2023 | Student Assistant | San Jose Academy of Bulacan |

## AFFILIATIONS

| Inclusive Dates | Name of Organization | Position |
|---|---|---|
| August 2023-2024 | Intellitech | Member |
| August 2024-2025 | Intellitech | Member |
| August 2025-2026 | Intellitech | Member |

## SKILLS

| SKILLS | Level of Competency | Date Acquired |
|---|---|---|
| MS Office | Intermediate | June 2011 |
| Java Programming | Intermediate | October 2023 |
| C# Programming | Intermediate | May 2024 |

## TRAININGS, SEMINARS OR WORKSHOP ATTENDED

| Inclusive Dates | Title of Training, Seminar or Workshop |
|---|---|
| **August 2023** | **National Service Training Program** |
| **February 2024** | **GitGud** |
| | **CodeFest** |

**Curriculum Vitae of**

# Gallano, Adrian V.

**1067 Malaya Street, Sapang Alat Road, Barangay 181, Pangarap Village, Caloocan City, NCR**
**gallano.360632@sjdelmonte.sti.edu.ph**
**+63927208694**

## EDUCATIONAL BACKGROUND

| Level | Inclusive Dates | Name of school/ Institution |
|---|---|---|
| Tertiary | August 2023 | STI College San Jose Del Monte |
| Vocational/Technical | | |
| High School | July 2017-2023 | Immaculada Conception Colleges |
| Elementary | August 2011-2017 | Our Lady of Mercy School |

## PROFESSIONAL OR VOLUNTEER EXPERIENCE

| Inclusive Dates | Nature of Experience Job Title | Name and Address of Company or Organization |
|---|---|---|

## AFFILIATIONS

| Inclusive Dates | Name of Organization | Position |
|---|---|---|
| August 2023-2024 | Intellitech | Member |
| August 2024-2025 | Intellitech | Member |
| August 2025-2026 | Intellitech | Member |

## SKILLS

| SKILLS | Level of Competency | Date Acquired |
|---|---|---|
| MS Office | Intermediate | June 2018 |
| Java Programming | Intermediate | October 2023 |
| C# Programming | Intermediate | September 2024 |

## TRAININGS, SEMINARS OR WORKSHOP ATTENDED

| Inclusive Dates | Title of Training, Seminar or Workshop |
|---|---|
| August 2023 | National Service Training Program |
| February 2024 | GitGud |